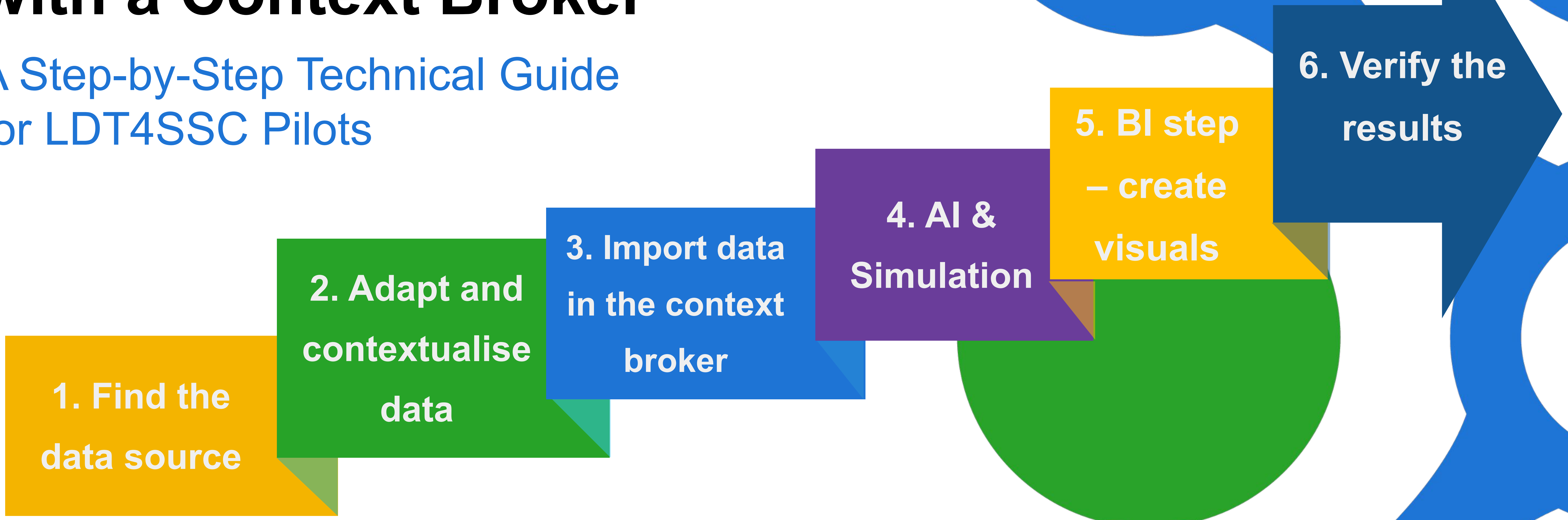
Prototyping your Use Case with a Context Broker

A Step-by-Step Technical Guide for LDT4SSC Pilots



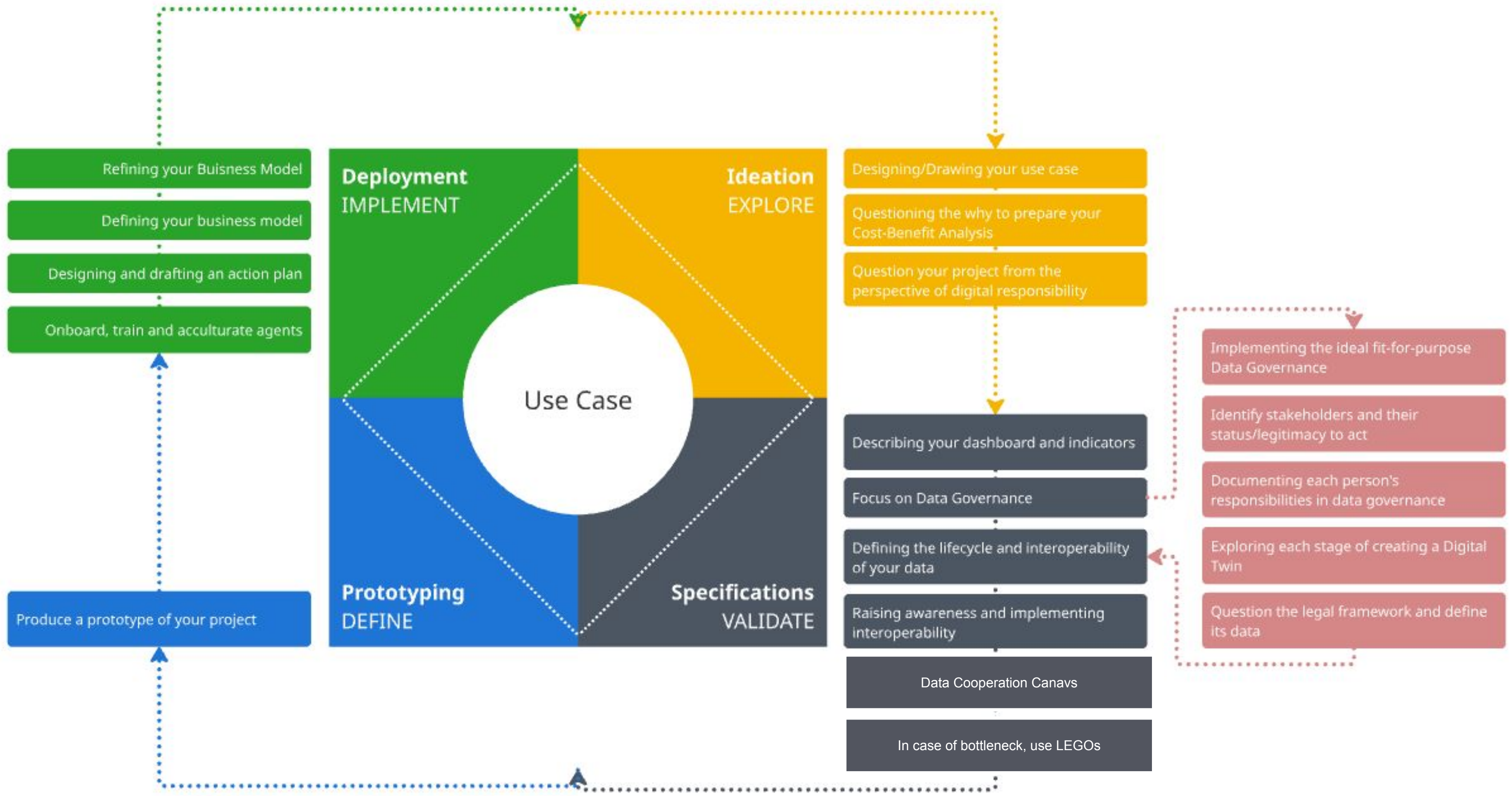
Introduction

LDT4SSC Methodology

This deck of slides serves two purposes:

- Present **an example** of how the **DEFINE (Prototyping) step** of the LDT4SSC Methodology can be carried-out.
- Present how a context broker can be used in a LDT architecture.

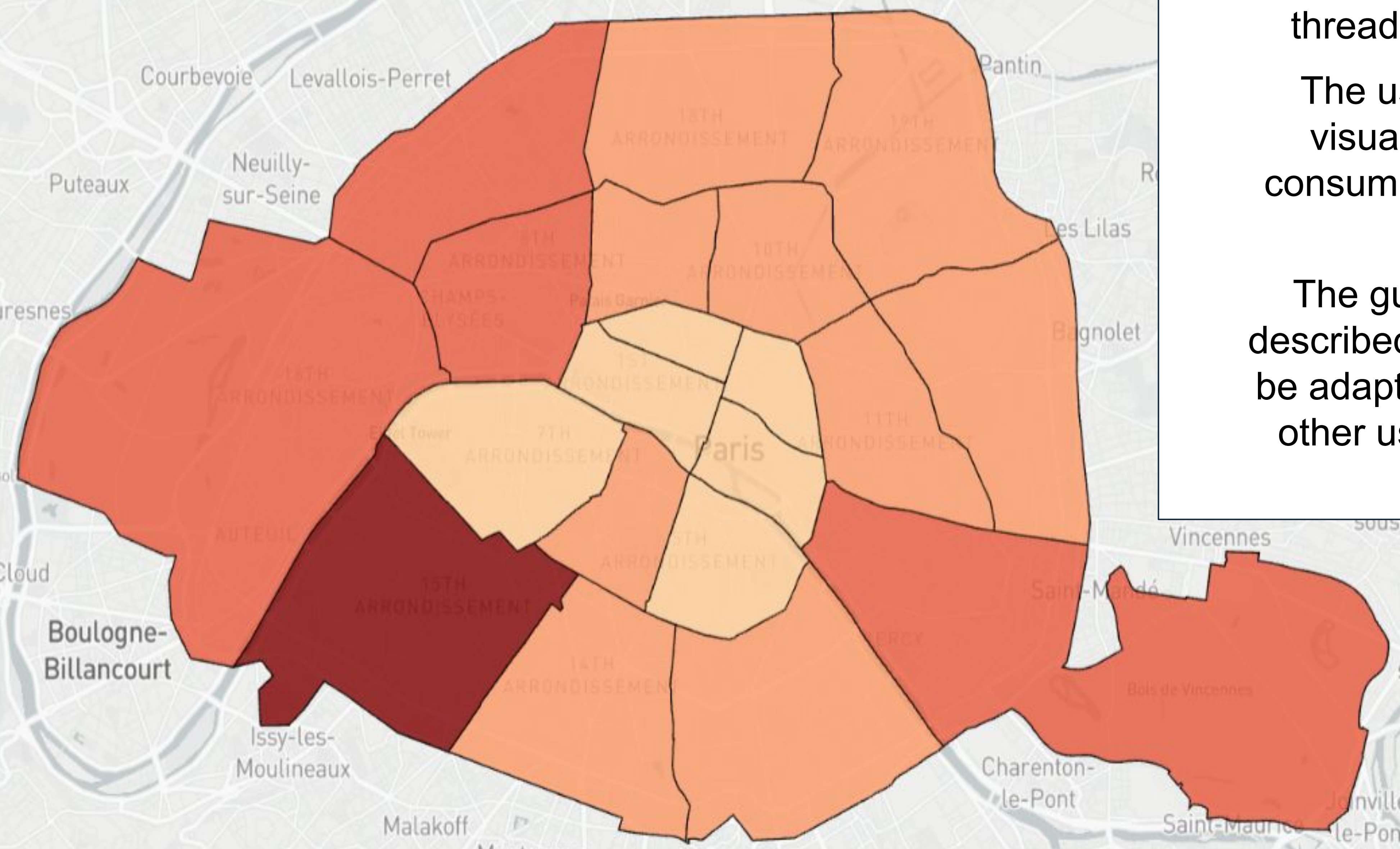
Following this step-by-step technical guide is **not mandatory** for pilots to complete this steps of the LDT4SSC Methodology. Rather, it is intended to serve as a **useful resource for pilots wishing to have a context broker** in their LDT architecture.



Example of a simple use case

Energy Consumption in the City of Paris

Carte des consommations à l'arrondissement



Within the context of these slides, a use case is presented as a read thread to illustrate the steps.

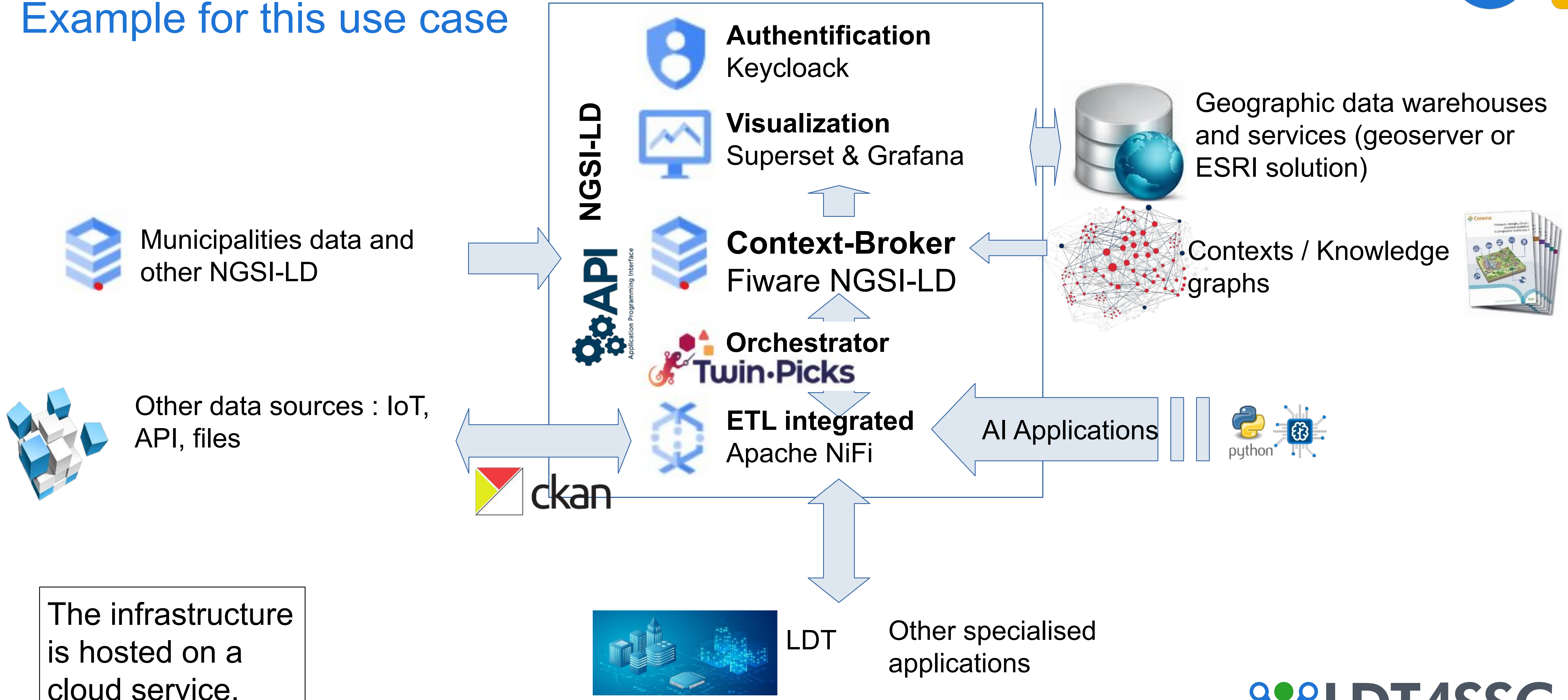
The use-case concerns the visualisation of the energy consumption of city of Paris per borough.

The guidance and approach described in this deck is meant to be adaptable and transferable to other use cases and technical architectures.

LDT4SSC

Cerema’s Platform architecture

Example for this use case



The infrastructure is hosted on a cloud service.

LDT4SSC

Local Digital Twins for Smart Sustainable Communities

1. Find the data source(s)

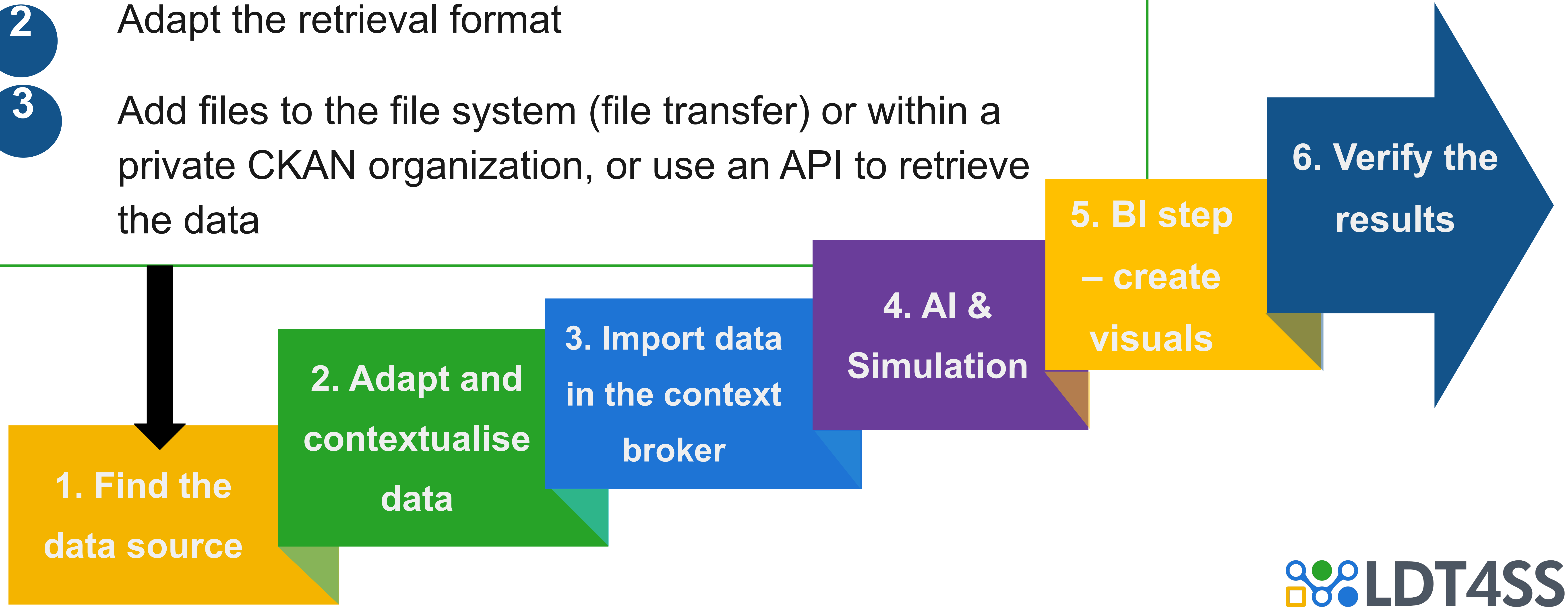
Funded by
the European Union

3. Prototyping

1. Find the data source(s)

Energy Consumption in the City of Paris

1. List the data sources
2. Adapt the retrieval format
3. Add files to the file system (file transfer) or within a private CKAN organization, or use an API to retrieve the data



LDT4SSC

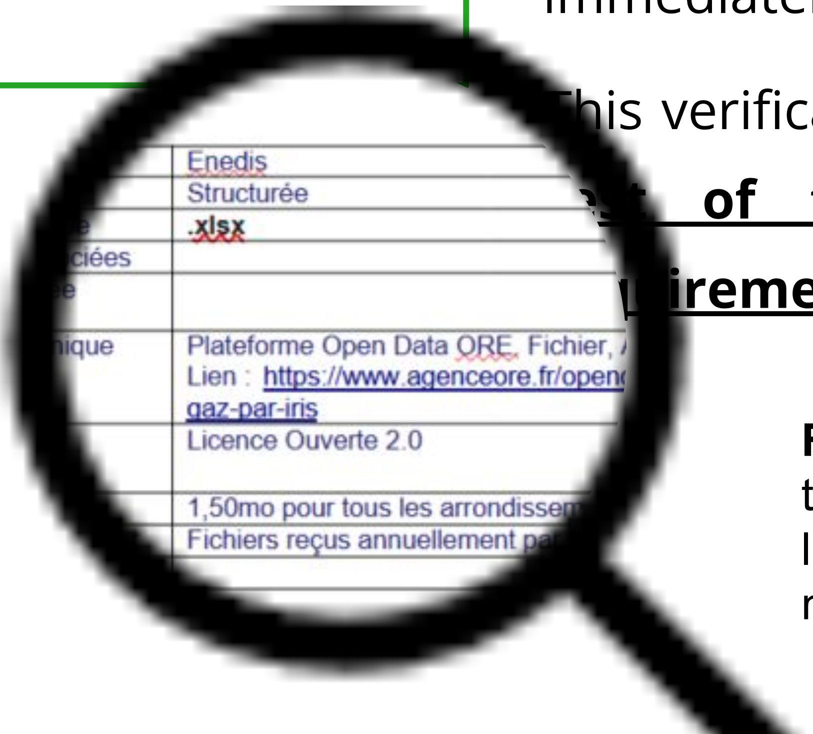
1. Find the data source

Energy Consumption in the City of Paris

3. Prototyping

1. List the data sources
2. Adapt the retrieval format
3. Add files to the file system (file transfer) or within a private CKAN organization, or use an API to retrieve the data

1. Find the data source(s)



The first step is to **verify the availability and quality of the data** needed in the specifications of the LDT use case. It is necessary to **ensure that each listed source is accessible**, that the provided links and/or endpoints work correctly, and that the formats (CSV, JSON, API, etc.) are usable. Each link/endpoint must be tested individually to confirm effective access to the data and its compliance with the expected format.

In case of an inactive link, an incompatible format, or a missing source, the information must be reported immediately for correction.

This verification **ensures that the database used for the rest of the project is reliable and meets the requirements of the use case**.

For each dataset describe in a table: Name, producer, type, format, metadata, schema, technical accessibility, license, volume, update frequency, quality and reliability, need for transformation or enhancement.

At the end of the **VALIDATE (Specification)** step, a specifications document/Terms of reference (TOR) setting out the expectations and requirements for the LDT was developed.

LDT4SSC

1. Find the data source

Energy Consumption in the City of Paris

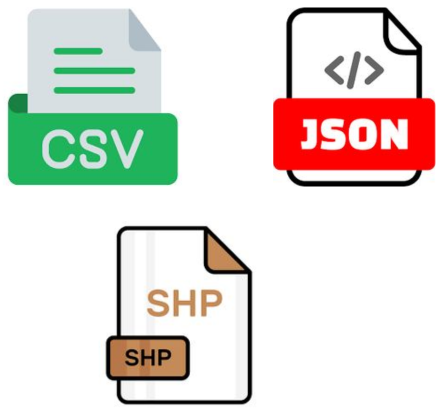
3. Prototyping

1. List the data sources
2. Adapt the retrieval format
3. Add files to the file system (file transfer) or within a private CKAN organization, or use an API to retrieve the data

1. Find the data source(s)

*: **BitVise** (SFTP) is used **to securely transfer files to an internal server, CKAN to publish and share open data via a dedicated platform, and APIs for dynamic and automated retrieval from external sources.**

CSV, SHP, and JSON format files must be **imported into BitVise*** to be accessible from the server environment and utilized by Apache NiFi. This step ensures that **all source data is centralized on the server and ready to be integrated** into the ingestion flow. It is recommended to **verify the successful import** of files into the dedicated directories before proceeding, in order to avoid any reading or path errors during the NiFi flow configuration.



For data sources accessible via an **API, the import step on BitVise is not necessary**. The calls can be made directly from Apache NiFi, which handles the real-time retrieval and processing of data. We thus move directly to the configuration of the NiFi flow to establish the connection with the API, define the call parameters (URL, authentication, update frequency), and integrate the data into the ingestion process.

LDT4SSC

1. Find the data source

Energy Consumption in the City of Paris

1. List the data sources
2. Adapt the retrieval format
3. Add files to the file system or use an API to retrieve the data

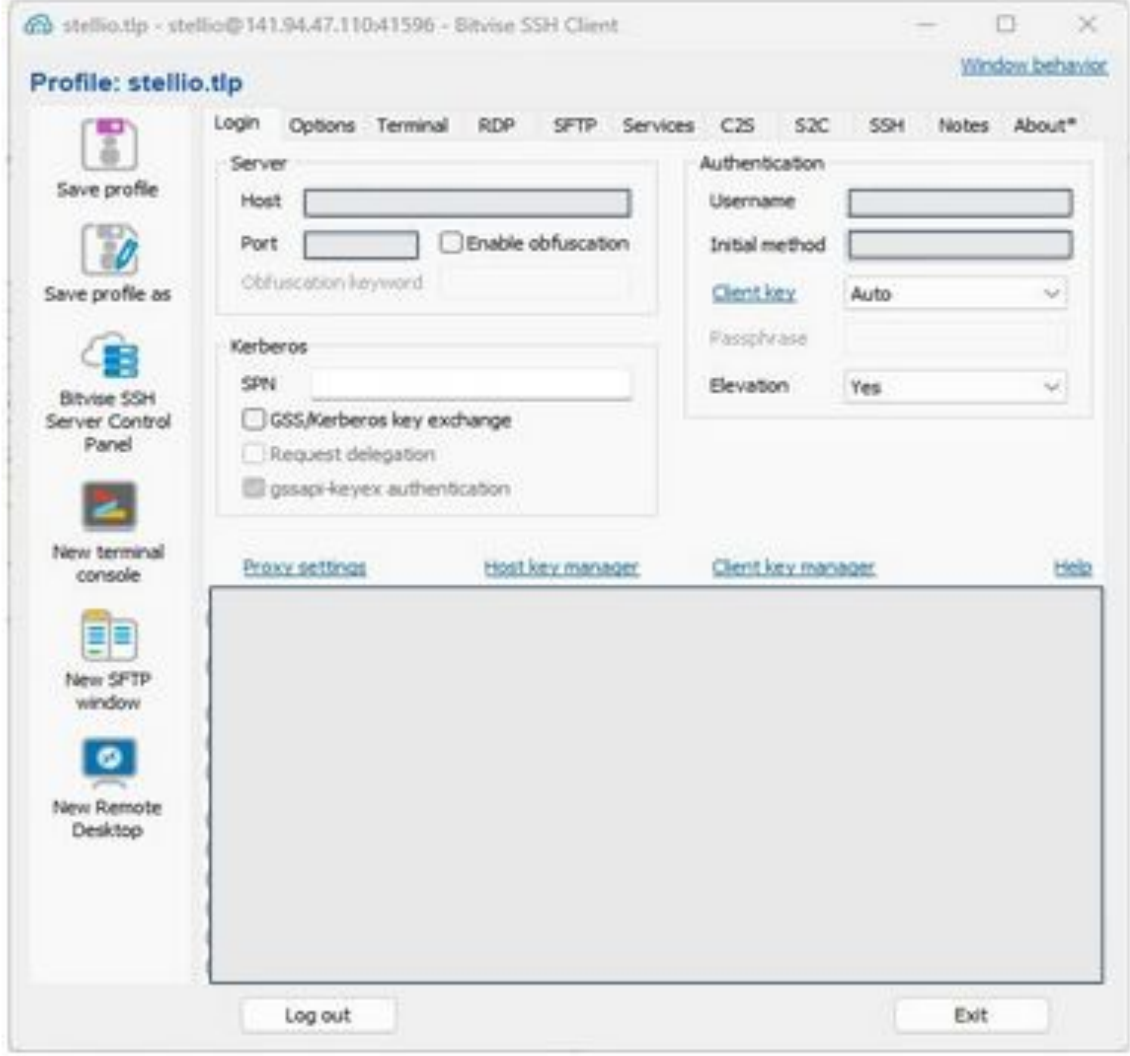
1. Find the data source(s)

Depending on the **nature and accessibility of the identified data sources**, **several methods** can be used to make them **available within the context broker**. Common solutions include:

- **file transfer** via tools such as **BitVise** (SFTP),
- the use of **open data management platforms** (e.g., CKAN),
- or **direct API calls** for dynamic retrieval.

The choice of method will depend on technical and security constraints, as well as the real-time needs of the project. In the case of a transfer via BitVise, the **connection to BitVise** must be made using a **profile that has the necessary access** related to the context broker environment. This ensures the necessary **rights to transfer, read, and manipulate the files** intended for ingestion into the data core.

Once the profile is selected, it is important to verify that the SFTP connection is established correctly and that the destination directories corresponding to the context broker are properly accessible before any file import.



Once connected to BitVise, **access the SFTP tab** to transfer the previously formatted files to the directory dedicated to the context broker. This transfer makes the **data available on the server** for subsequent use in Apache NiFi. It is advisable to **verify that the files are correctly placed in the right folders** and that their **names follow the expected convention**, in order to ensure a smooth integration during the creation of the NiFi flow.

Local Digital Twins for Smart Sustainable Communities

2. Adapt and contextualise the data

Funded by
the European Union

2. Adapt and contextualise data

Energy Consumption in the City of Paris

1. List the key variables to keep
2. Search for existing ontologies for the data
3. Make the ontology usable
4. Update the context based on the new variables
5. If needed, adapt or create new ontologies and publish them

1. Find the data source
2. Adapt and contextualise data
3. Import data in the context broker
4. AI & Simulation
5. BI step – create visuals
6. Verify the results

2. Adapt and contextualise data

3. Prototyping

From data to knowledge

LDT4SSC

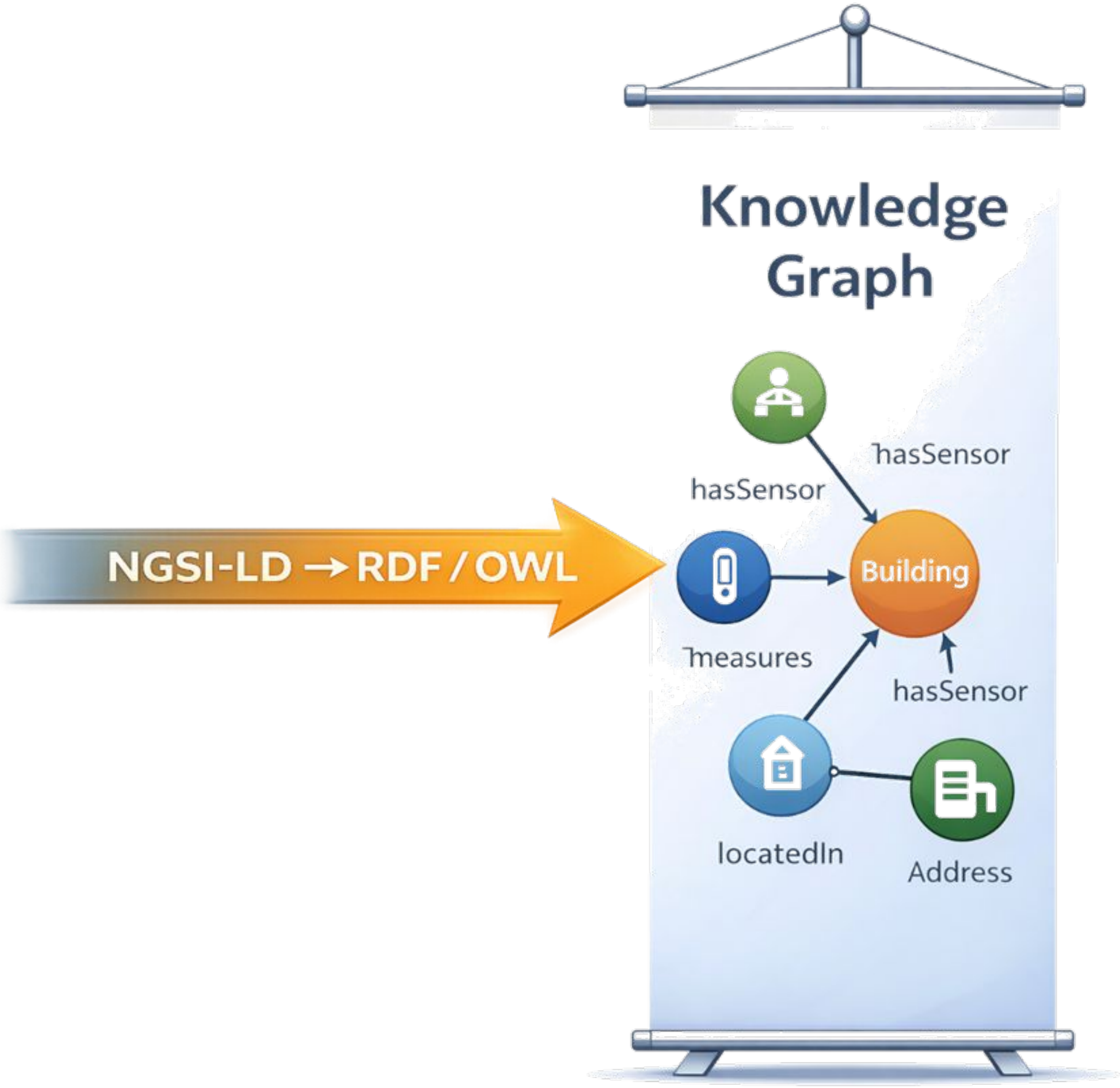
2. Adapt and contextualise data

NGSI-LD MAPPING

3. Prototyping

TABLE

ID	Name	Type	Location
1	Building1	Building	???
2	SensorA	Sensor	Paris
3	SensorB	Sensor	Lyon



2. Adapt and contextualise data

3. Prototyping

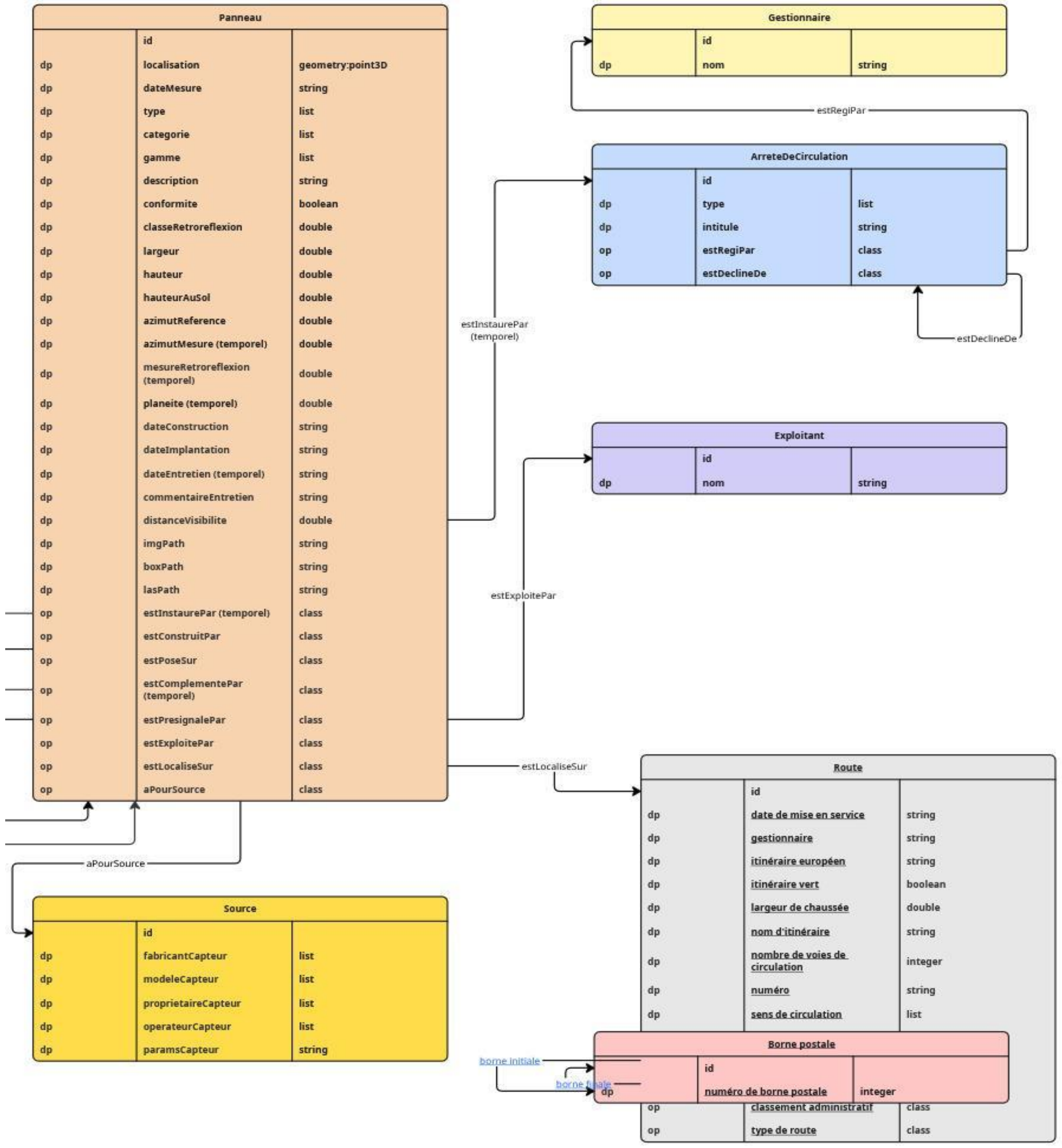
Energy Consumption in the City of Paris

1 List the key variables to keep

During the **modeling phase** conducted with the end user, the **variables relevant to obtain the final output must be identified**. These variables can correspond to **specific fields or relationships between multiple data sources**. The objective is to **precisely define the elements that will be contextualized and integrated** into the context broker. This step ensures that only the information necessary for the visualisations and final indicators is retained. If no modeling has yet been conducted in consultation with all project stakeholders, it is imperative to do so before proceeding, in order to ensure a common understanding of the data structure and intended use*.

2. Adapt and contextualise data

*: see *step 5. Eventually, adapt or create new ontologies and publish them* (slides 13-14).



2. Adapt and contextualise data

3. Prototyping

Energy Consumption in the City of Paris

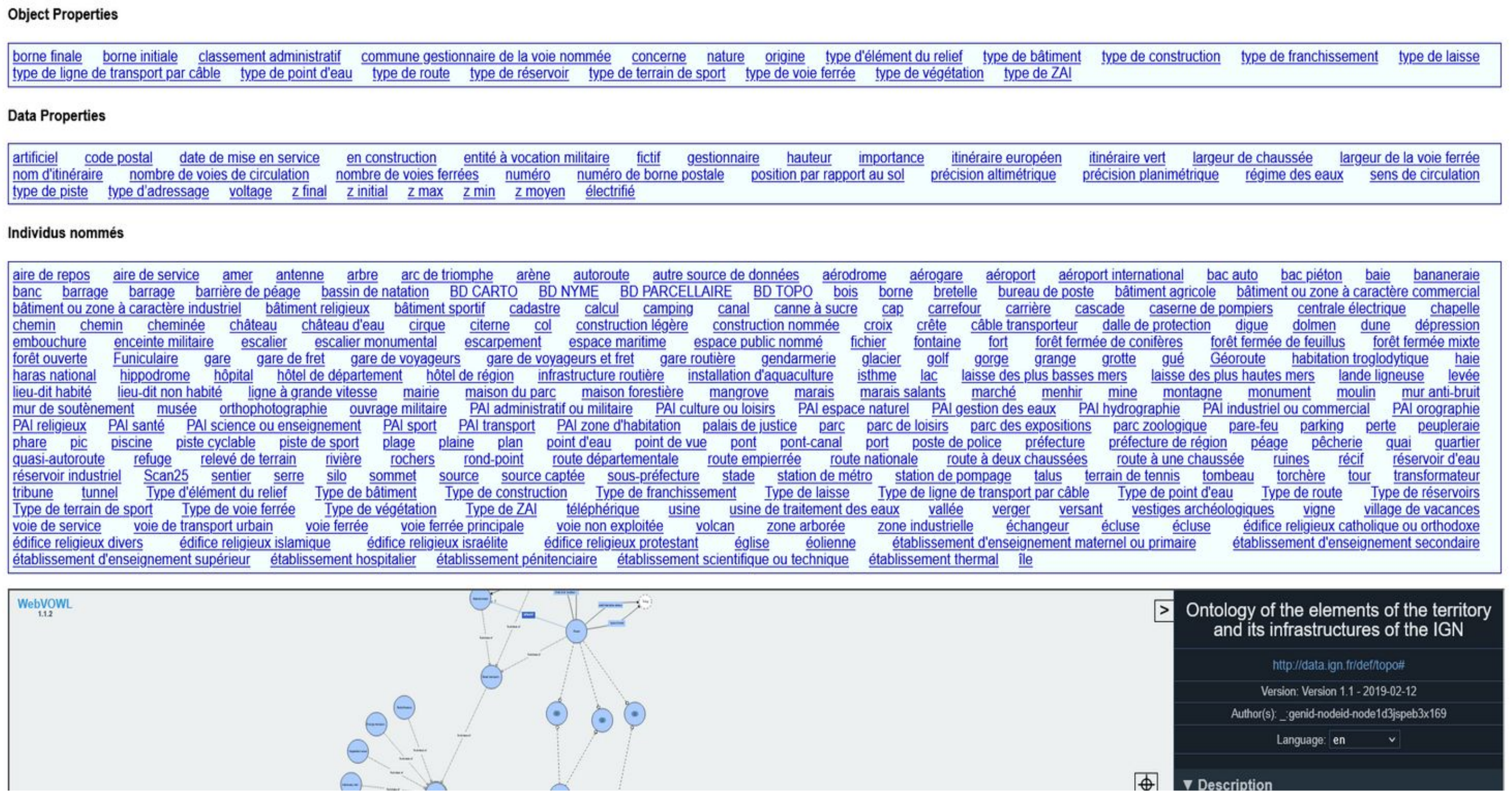
1. List the key variables to keep
2. Search for existing ontologies for the data

Once the complete list of fields is established, the next step is to **associate an ontology with each variable**. The goal is to **rely on existing ontologies** to standardize the data and ensure their compatibility with the NGSI-LD model. For this, it is recommended to search for already recognized ontologies in the relevant domain, or, failing that, to create new ones for variables that do not find an equivalent.

2. Adapt and contextualise data

The search can rely on domain knowledge or be assisted by a high-performing artificial intelligence agent capable of suggesting relevant matches. However, **each suggestion must absolutely be manually verified** to ensure that the chosen ontology correctly describes the meaning and role of the field in the context of the use case.

here is for example the IGN ontology: https://data.ign.fr/def/topo/20190212.htm



2. Adapt and contextualise data

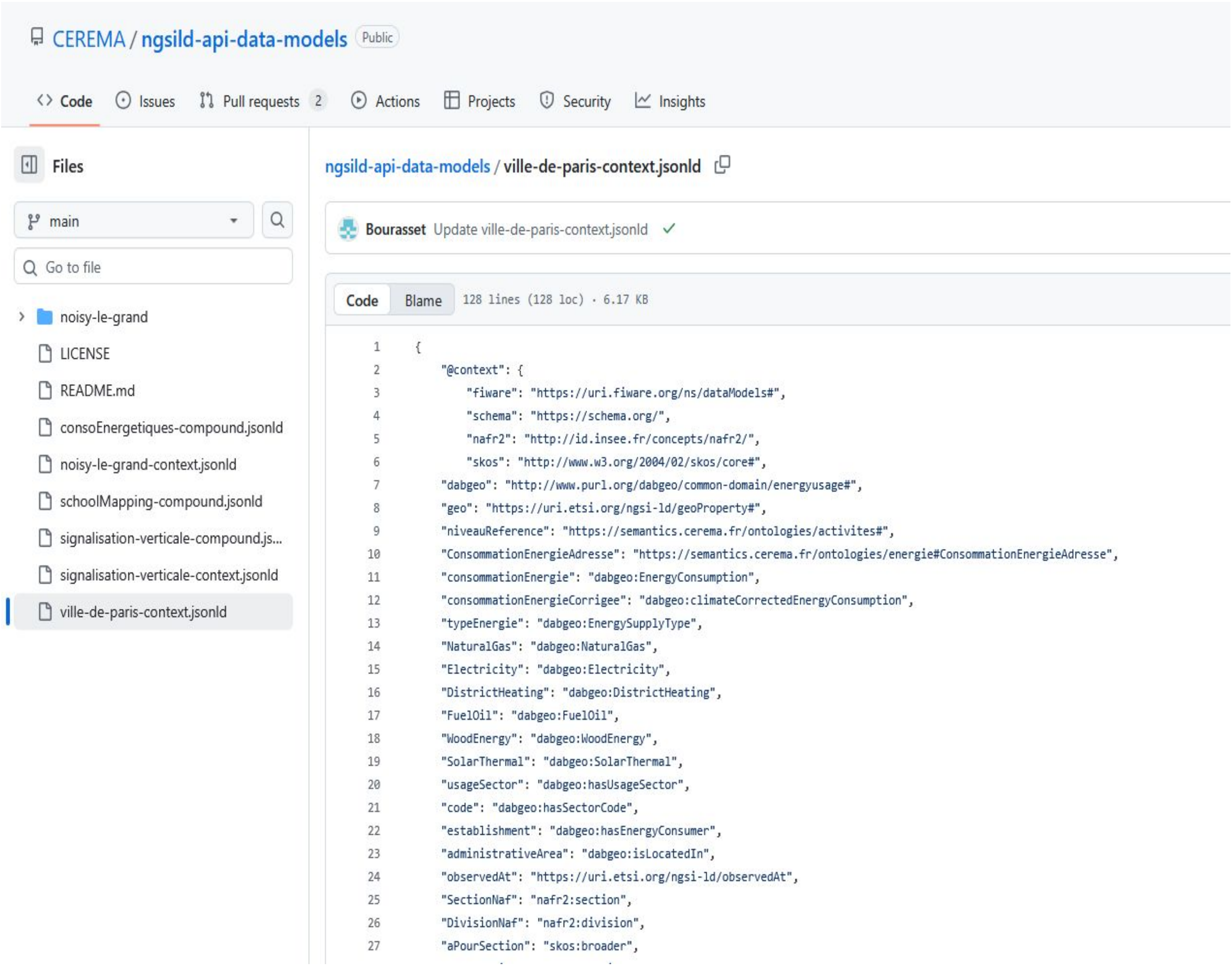
3. Prototyping

Energy Consumption in the City of Paris

1. List the key variables to keep
2. Search for existing ontologies for the data
3. Ensure the ontology is usable / make it usable

It is **absolutely essential that the context you create** (with your fields + ontologies) **is placed in the correct repository** / directory **and in the correct JSON-LD format** so that it can be used by the tools (notably the context broker / NGSI-LD). The GitHub repository "*CEREMA/ngsild-api-data-models*" precisely serves as a library of shared JSON-LD contexts.

2. Adapt and contextualise data



LDT4SSC

2. Adapt and contextualise data

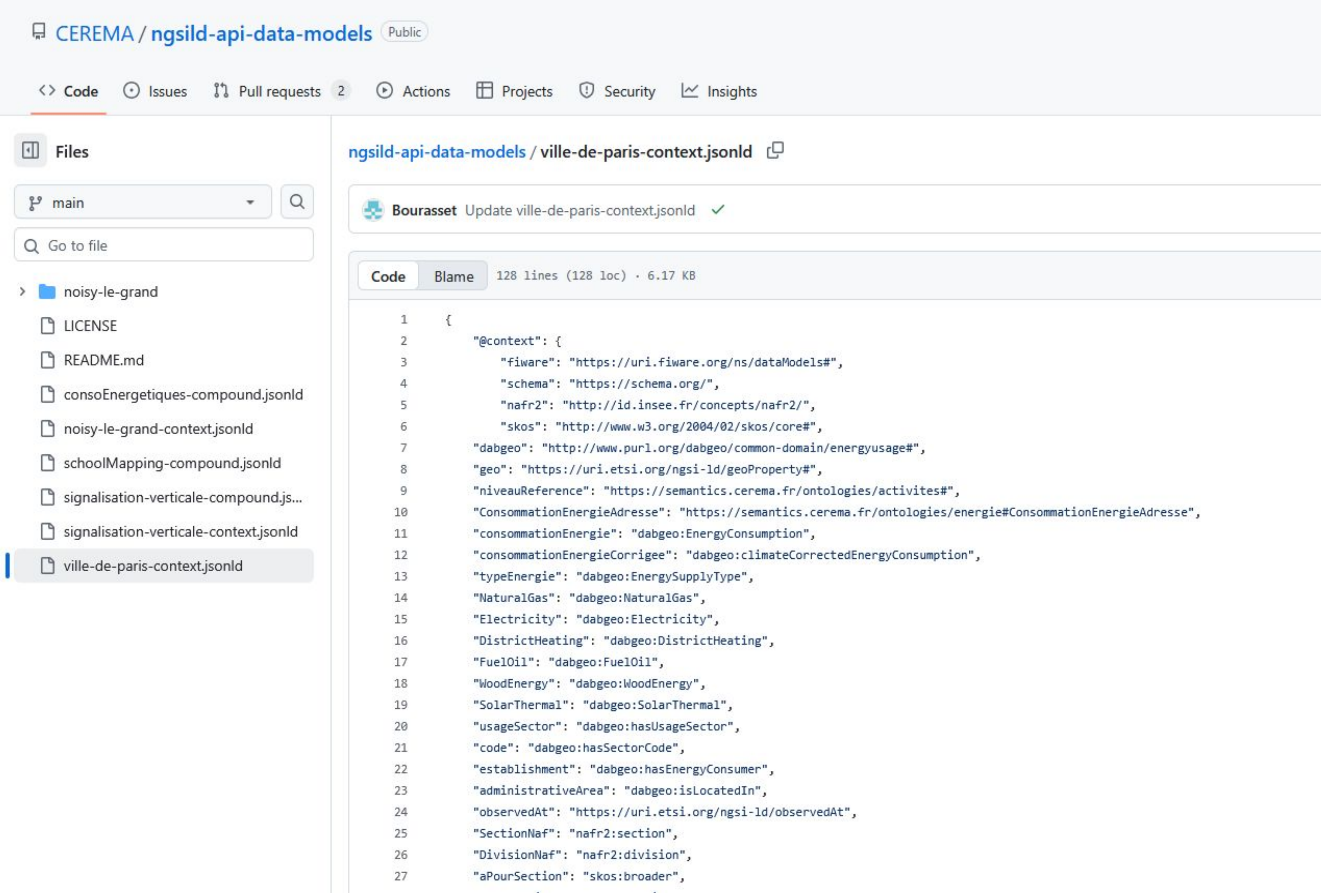
Energy Consumption in the City of Paris

3 Make the ontology usable

The ***CEREMA/ngsild-api-data-models*** GitHub repository serves as a **central repository for all use cases addressed in the platform and their associated contexts**. Each use case is documented there in the form of two main files: **a compound file** and **a context file**, both in *.jsonld format* to comply with the NGSI-LD standard.

When **creating a new context**, it is recommended to **rely on an existing model already present in the repository**. This allows for consistency in structure and naming across different use cases. The **context file** contains the **definition of entities, attributes, and relationships, as well as links to the associated ontologies**. It must be written with rigor, adhering to the JSON-LD syntax and the conventions already established in the repository.

Once the new context is created, it must be added to the correct directory of the GitHub repository, following the nomenclature of the other use cases. This step ensures that the context will be recognized and usable by CEREMA's tools and by the Stellio context broker.

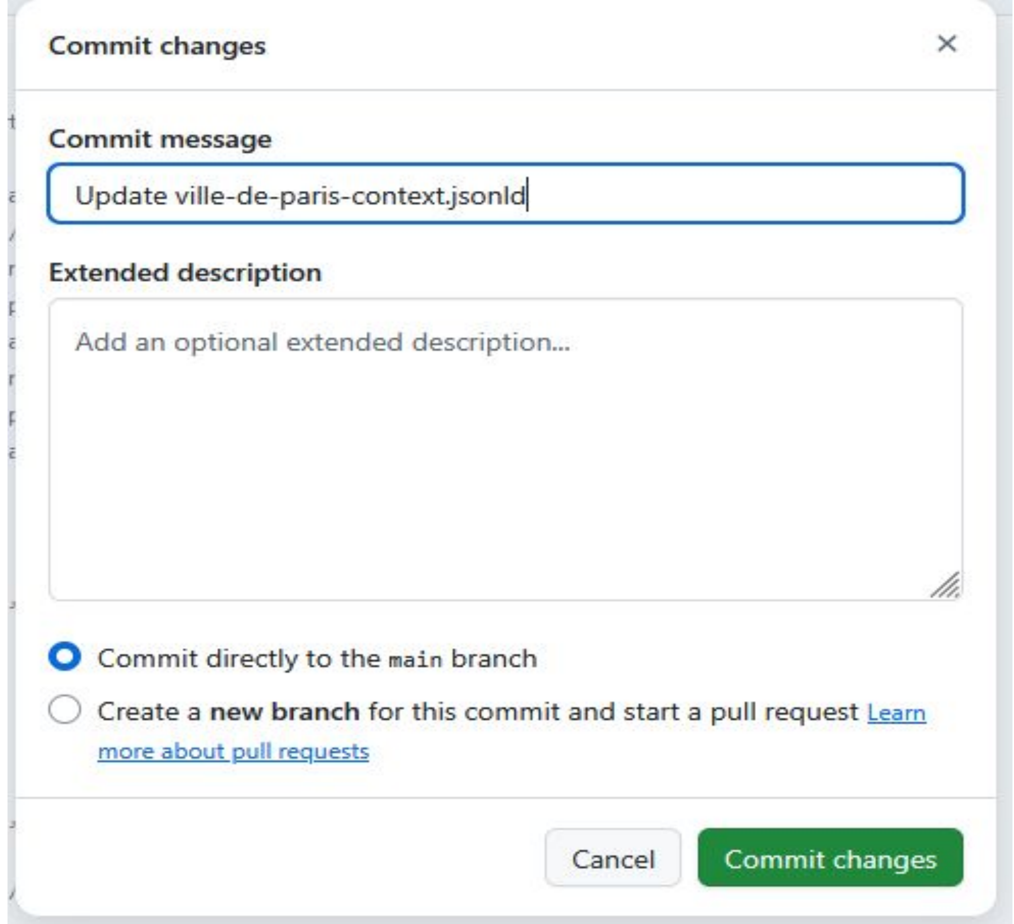


2. Adapt and contextualise data

3. Prototyping

Energy Consumption in the City of Paris

1. List the key variables to keep
2. Search for existing ontologies for the data
3. Make the ontology usable
4. Update the context based on the new variables
5. Eventually, adapt or create new ontologies and publish them



2. Adapt and contextualise data

If **new variables need to be added during the project**, the principle remains the same: simply **modify the context file** corresponding to the use case in the GitHub repository. You must then **add a new line describing the ontology associated with the variable**, following the same structure and format as the existing entries. Once the modifications are made, it is essential to commit the changes with a clear and explicit message describing the additions or adjustments made. This good practice ensures traceability and historical tracking of all context changes, thereby facilitating maintenance and understanding for other contributors.

2. Adapt and contextualise data

3. Prototyping

Energy Consumption in the City of Paris

1. List the key variables to keep
2. Search for existing ontologies for the data
3. Make the ontology usable
4. Update the context based on the new variables
5. Eventually, adapt or create new ontologies and publish them

2. Adapt and contextualise data

If **no existing ontology matches your variable**, it is necessary to **create** a new one **following the standard format** used by CEREMA, while making sure to describe all concepts (entities, properties, relationships). Each new ontology must be **defined in the form of a URI hosted on the official domain: https://semantics.cerema.fr/ontologies/**. This URL serves as a **unique reference to identify the resource in the NGSI-LD ecosystem**. The name of the ontology must be clear, explicit, and compliant with the current naming conventions (camelCase, without spaces or special characters). Once the ontology is created, it can be added to the corresponding context file in the GitHub repository to be recognized and utilized by all the tools.

```
"ConsommationEnergieAdresse": "https://semantics.cerema.fr/ontologies/energie#ConsommationEnergieAdresse",
```

```
"niveauReference": "https://semantics.cerema.fr/ontologies/activites#",
```

LDT4SSC

2. Adapt and contextualise data

3. Prototyping

Energy Consumption in the City of Paris

5 Create a new ontology

```
"ConsommationEnergieAdresse": "https://semantics.cerema.fr/ontologies/energie#ConsommationEnergieAdresse",
```

This format does indeed **allow for the creation of new ontologies** when necessary, but it should be **<u>used sparingly</u>**. The **<u>priority should always be to reuse an existing ontology to ensure consistency and interoperability of data across different use cases</u>**. The creation of a new ontology should only be considered as a last resort, when no existing resource covers the functional need. This approach ensures better compatibility with other models and limits the proliferation of redundant definitions in the ecosystem.

LDT4SSC

Local Digital Twins for Smart Sustainable Communities

3. Import Data into the context broker

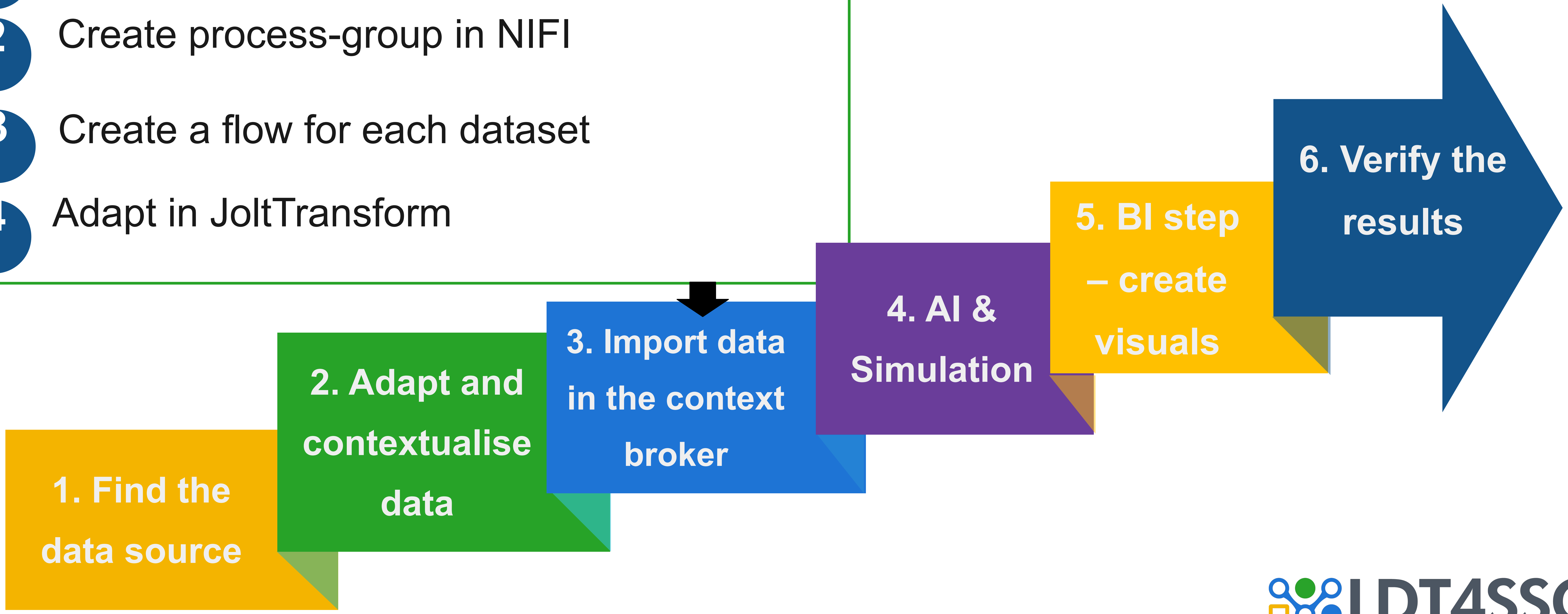
Funded by the European Union

3. Import data in the context broker

3. Prototyping

Energy Consumption in the City of Paris

1. Create a specific parameter
2. Create process-group in NIFI
3. Create a flow for each dataset
4. Adapt in JoltTransform



3. Import data in the Stellio context broker

3. Prototyping

Energy Consumption in the City of Paris

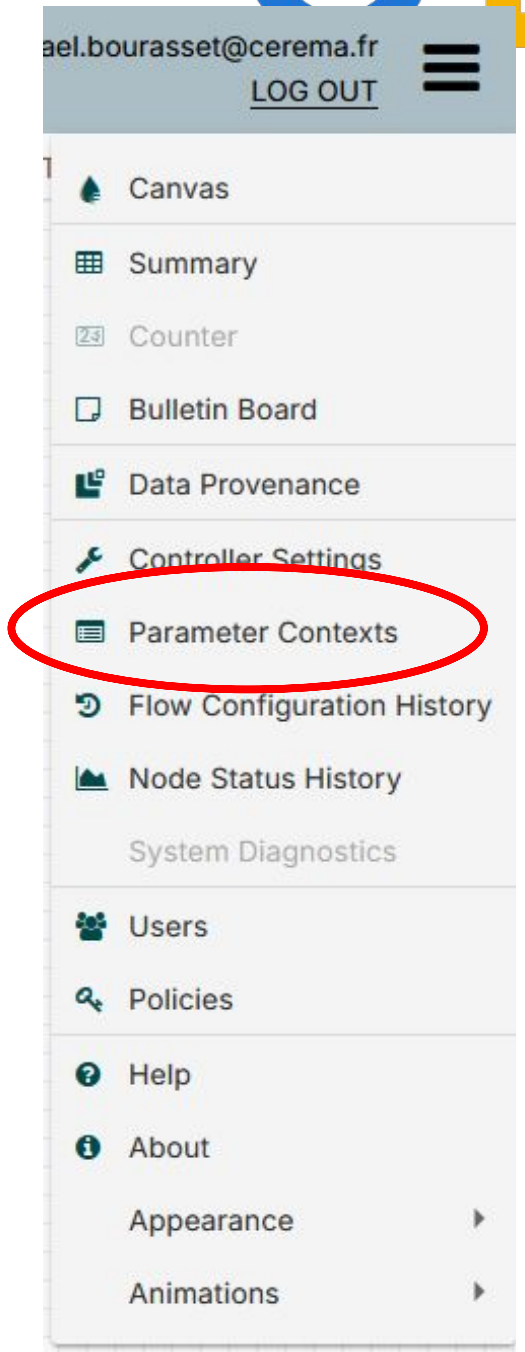
1

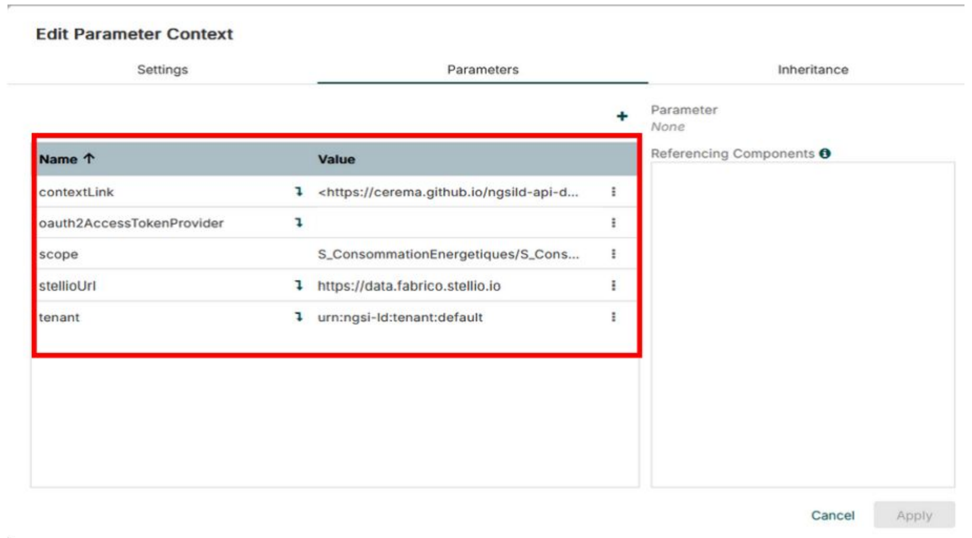
Create a parameter context

Before starting to work on the flows, it is necessary to **connect to the Apache NiFi interface** and **have the appropriate usage rights**. Once connected, one of the **first elements to configure is the <u>Parameter Context</u>**, which will be frequently used during the creation and management of your flows. This parameter allows you to **assign the same logical context to all the flows that will inherit from it**, thereby ensuring the consistency of configurations and connections between them.

The **creation of a new Parameter Context** is done directly **from the NiFi navigation bar by clicking on the "+" symbol** located at the top of the page. You then simply need to enter the necessary information, drawing inspiration from the already existing contexts to maintain consistency in the names, keys, and values used. Although the procedure is simple, it is **strongly recommended to have the created context checked by a colleague or an administrator** to ensure that no configuration errors will disrupt future flows.

3. Import data in the Stellio context broker





3. Import data in the Stellio context broker

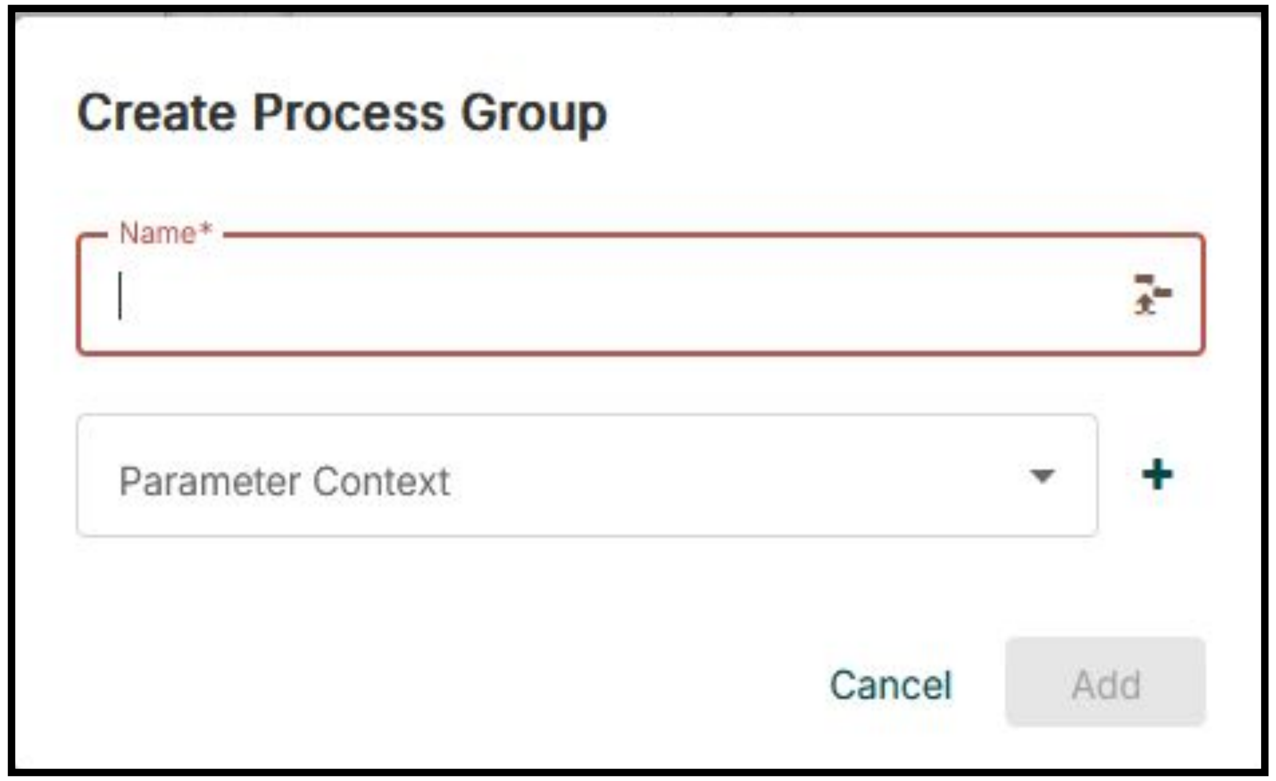
Energy Consumption in the City of Paris

1. Create a parameter context
2. Create process-group in NIFI

A **Process Group** in Apache NiFi acts as **a logical folder that allows you to group and organize multiple processing flows**. It facilitates the structuring of work by separating the different use cases or stages of the same project. When creating it, it is **essential to give it an explicit name** reflecting its content as well as a Parameter Context so that all the components it contains inherit the correct parameters.

The creation is simply **done by dragging and dropping the Process Group icon from the toolbar to the workspace**, then defining its name and associated context. This visual approach makes NiFi intuitive and allows for efficient organization of flows according to the project's logic.

3. Import data in the Stellio context broker



3. Import data in the Stellio context broker

3. Prototyping

Energy Consumption in the City of Paris

1. Create a parameter context
2. Create process-group in NIFI
3. Create a flow for each dataset

The **complexity of a NiFi flow** directly depends on the **expected level of transformation** and the **nature of the processed data**. In some cases, the flow can be very short and linear, limited to a few processors to import, reformat, and send the data to the context broker.

In others, it **can become more complex**, integrating steps of cleaning, filtering, joining, or advanced enrichment. The important thing is to **adapt the flow structure to the actual needs of the use case**, prioritizing clarity and maintainability over technical complexity.

3. Import data in the Stellio context broker

Creating a flow in Apache NiFi requires a good understanding of the tool. It is therefore strongly recommended to follow tutorials well as the technical documentation before starting. These resources allow you to master the basics of NiFi and avoid common mistakes when designing complex flows.

In the context of our use cases, **a flow primarily aims to transform raw data, previously prepared and imported via BitVise**, to make it **compatible with the NGSI-LD format for integration into the Stellio Context Broker**. The flow thus ensures a complete processing chain: ingestion, transformation, contextualization, and sending.

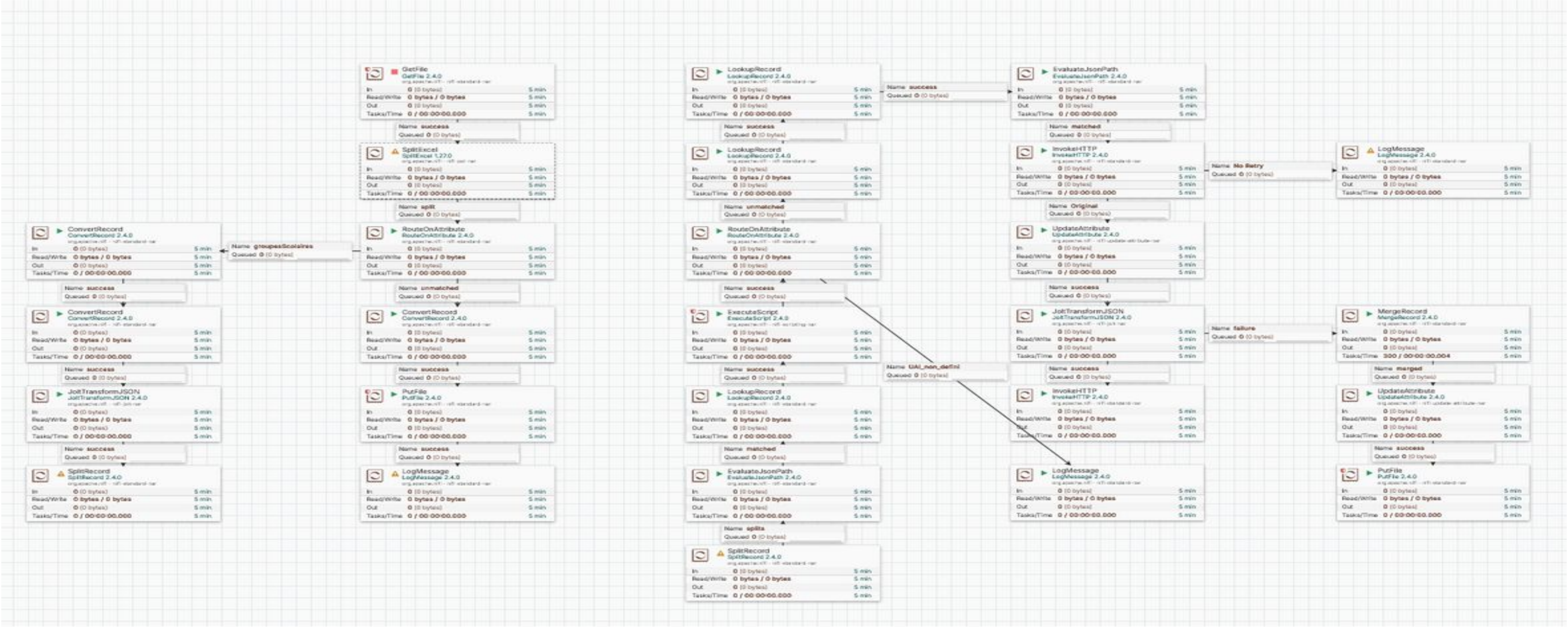
This step requires a certain **mastery of SQL, Groovy, and JOLT**, these languages being essential for carrying out the transformations, enrichments, and mappings necessary for compliance with the NGSI-LD model. The expected result is a **robust, automated flow perfectly aligned with the structure of the previously defined context**.

3. Import data in the Stellio context broker

3. Prototyping

Energy Consumption in the City of Paris

3 Create a flow for each dataset



Example of a flow

LDT4SSC

3. Import data in the Stellio context broker

3. Prototyping

Energy Consumption in the City of Paris

1. Create a parameter context
2. Create process-group in NIFI
3. Create a flow for each dataset
4. Adapt in JoltTransform

3. Import data in the Stellio context broker

When creating your flow, you will need to **configure a JoltTransform**, an essential step to **transform the structure of your data into the format expected by the Stellio Context Broker**. Jolt allows you to **apply JSON transformation rules in a declarative manner**, facilitating the transition from a source model to the NGSI-LD model.

To **test and validate your transformations** before integrating them into NiFi, it is highly recommended to use the following site: https://jolt-demo.appspot.com/#inception. This online tool allows you to quickly experiment with your Jolt specifications and visualize the expected result.
Don't hesitate to consult Jolts already implemented on other flows to inspire you with best practices and adapt the logic to your own context.

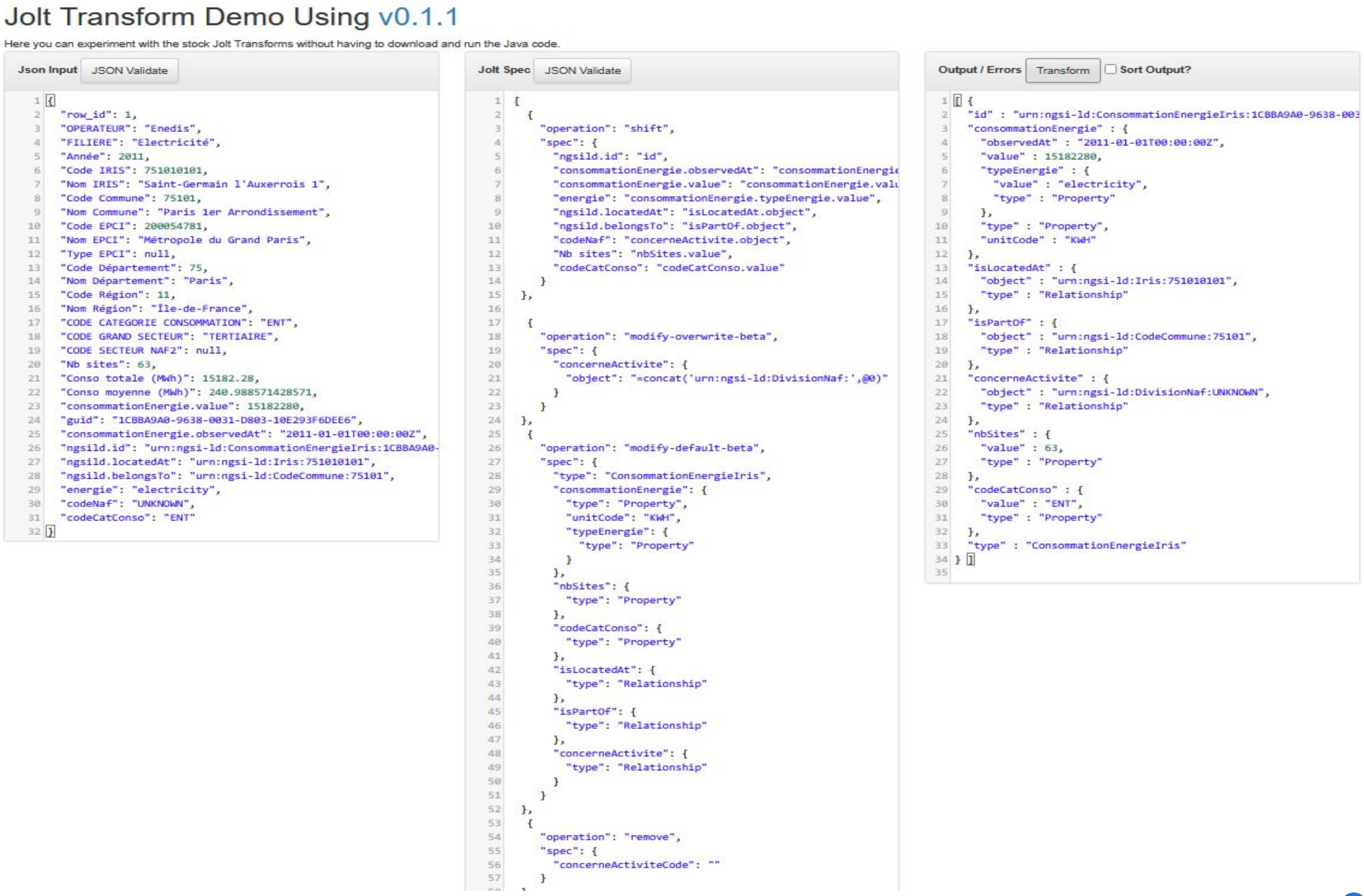
LDT4SSC

3. Import data in the Stellio context broker

3. Prototyping

Energy Consumption in the City of Paris

4 Adapt in JoltTransform



Example of a JoltTransorm process

LDT4SSC

Local Digital Twins for Smart Sustainable Communities

4. AI & Simulation : create services

Funded by
the European Union

4. AI & Simulation : create services

3. Prototyping

Territorial Digital Twin use cases

1. Define the decision-support objective
2. Prepare the analytical datasets
3. Train, calibrate or configure the model
4. Publish AI outputs back into the digital twin
5. Expose results

1. Find the data source
2. Adapt and contextualise data
3. Import data in the context broker
4. AI & Simulation
5. BI step – create visuals
6. Verify the results

LDT4SSC

4. AI & Simulation : create services

Territorial Digital Twin use cases

1. Define the decision-support objective
2. Prepare the analytical datasets
3. Train, calibrate or configure the model
4. Publish AI outputs back into the digital twin
5. Expose results

Once the data is contextualised and available in the context-broker, the digital twin can support predictive, prospective or simulation-based services.

This step turns the broker from a data integration layer into a decision-support layer: models generate forecasts, scenarios, risks or recommendations that can be stored, traced, compared and visualised.

4. AI & Simulation

LDT4SSC

4. AI & Simulation : create services

3. Prototyping

Territorial Digital Twin use cases

1. Define the decision-support objective
2. Prepare the analytical datasets
3. Train, calibrate or configure the model
4. Publish AI outputs back into the digital twin
5. Expose results

Recommended artefact

A mapping table:

→ source field
→ NGSI-LD entity/property
→ ontology
→ transformation rule
→ feature name
→ quality control.

AI models usually require an analytical dataset built from the entities stored in the context broker. This preparation step may include time aggregation, spatial joins, feature engineering, missing-value handling, and consistency checks.

The dataset must keep traceability to the source entities and to the transformation rules used to build it.

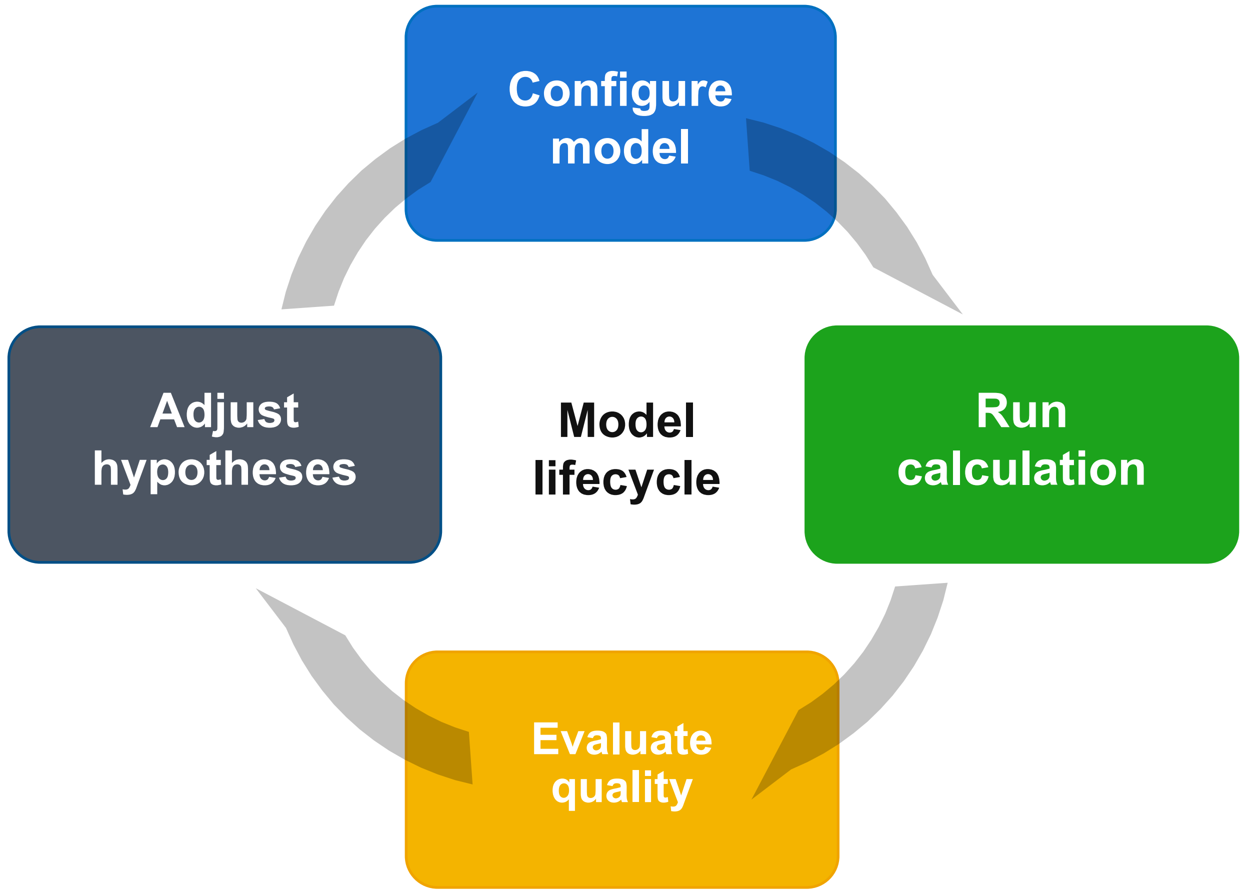
4. AI & Simulation

LDT4SSC

4. AI & Simulation : create services

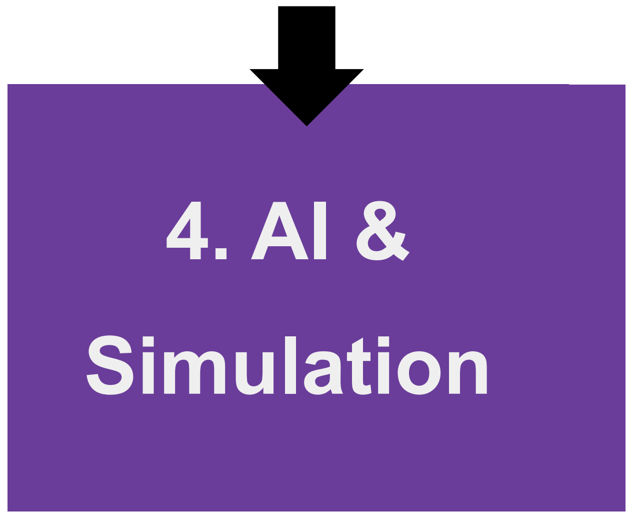
Territorial Digital Twin use cases

1. Define the decision-support objective
2. Prepare the analytical datasets
3. Train, calibrate or configure the model
4. Publish AI outputs back into the digital twin
5. Expose results



The model can be statistical, machine-learning based, simulation-based, rule-based, or hybrid. For territorial digital twins, the most useful approach is often not a black box: scenarios, hypotheses and calibration parameters must remain understandable.

The objective is to create a reproducible service, with explicit inputs, model version, assumptions and validation metrics.



LDT4SSC

4. AI & Simulation : create services

3. Prototyping

Territorial Digital Twin use cases

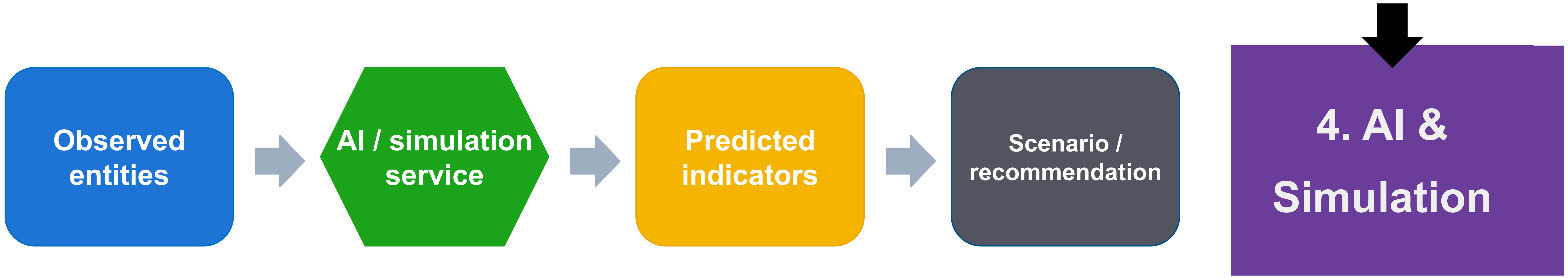
1. Define the decision-support objective
2. Prepare the analytical datasets
3. Train, calibrate or configure the model
4. Publish AI outputs back into the digital twin
5. Expose results

The outputs of AI or simulation services should be treated as first-class digital twin data. They can be written back to the context broker as predicted values, indicators, scenarios, recommendations or model execution records.

Each result should include metadata: timestamp, model version, assumptions, confidence level, and links to source entities.

Possible NGSI-LD output entities :

SimulationRun · Forecast · Scenario · RiskIndicator · Recommendation · ModelExecution



LDT4SSC

4. AI & Simulation : create services

3. Prototyping

Territorial Digital Twin use cases

1. Define the decision-support objective
2. Prepare the analytical datasets
3. Train, calibrate or configure the model
4. Publish AI outputs back into the digital twin
5. Expose results to dashboards / decision tools

Observed data
what happened

Scenarios
what if...

Predictions
what may happen

Decisions
what to do

Next: BI / visualisation

The BI step can then visualise both observed data and model outputs: forecasts, scenarios, differences between expected and actual values, uncertainty levels, and decision-support indicators.

The verification step must also include model-specific controls: reproducibility, bias or drift checks, confidence intervals, and traceability of hypotheses.

4. AI & Simulation

LDT4SSC

Local Digital Twins for Smart Sustainable Communities

5. Create visuals – BI step

Funded by
the European Union

5. BI steps – create visuals

Energy Consumption in the City of Paris

1. Import data in SUPERSET
2. Create the virtual datasets
3. Start to create the dashboard
4. Understanding filters and connection

1. Find the data source
2. Adapt and contextualise data
3. Import data in the context broker
4. AI & Simulation
5. BI step – create visuals
6. Verify the results

5. BI steps – create visuals

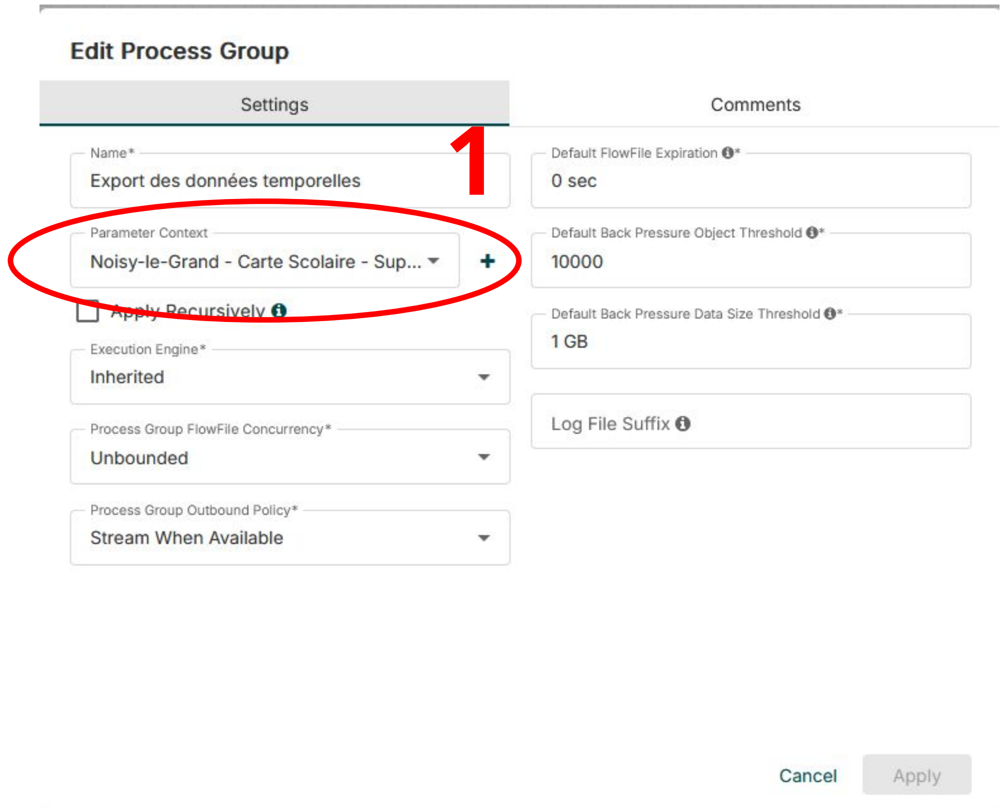
3. Prototyping

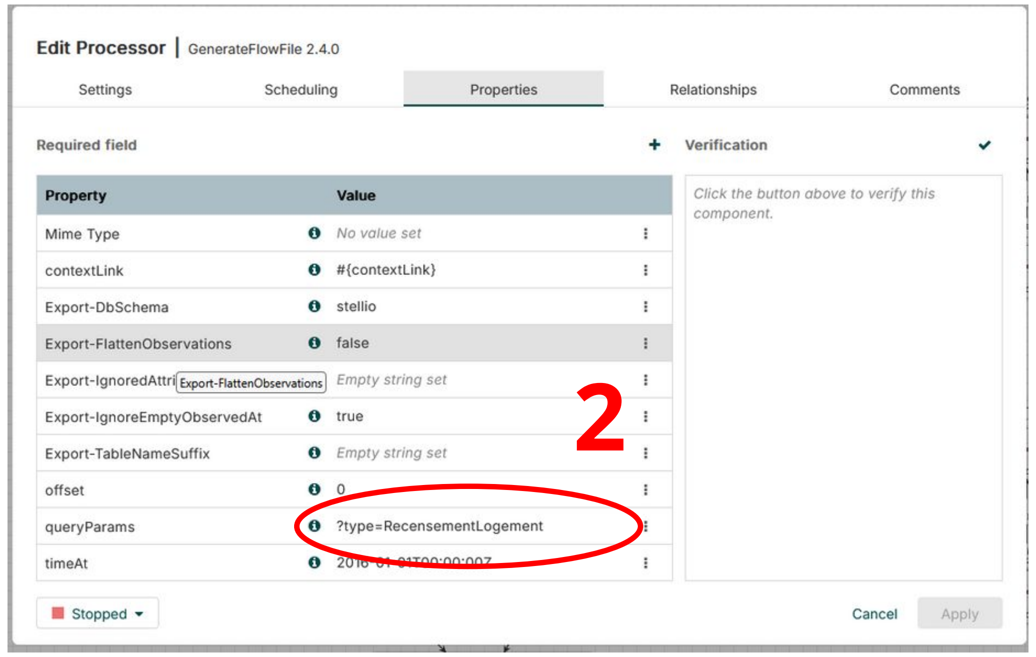
Energy Consumption in the City of Paris

1. Import data in SUPERSET
2. Create the virtual datasets
3. Start to create the dashboard
4. Understanding filters and connection

Once **your flow is executed** and your data is visible in Stellio via Postman, it becomes **usable in a BI visualization** tool such as Grafana. These tools allow for the **creation of dynamic and interactive dashboards** from the entities stored in the Context Broker.

If you wish to use other software, such as Apache Superset, it is necessary to **launch a specific NiFi flow, already configured and ready to use**: **"Export Superset LER"**. This flow allows for the **automatic export of data from Stellio to the database accessible by Superset**. Two versions exist: an export flow for temporal data, and an export flow for non-temporal data.





4. BI step – create visuals

Before launching this flow, two checks are essential:

1) The **Parameter Context must be correctly assigned** to your use case to ensure proper data routing.
2) The **precise type of data to be exported** (temporal or non-temporal) must be **correctly defined** to avoid any integration errors.

Once these parameters are validated, the flow can be launched, automatically feeding your Superset dashboards with data from Stellio.

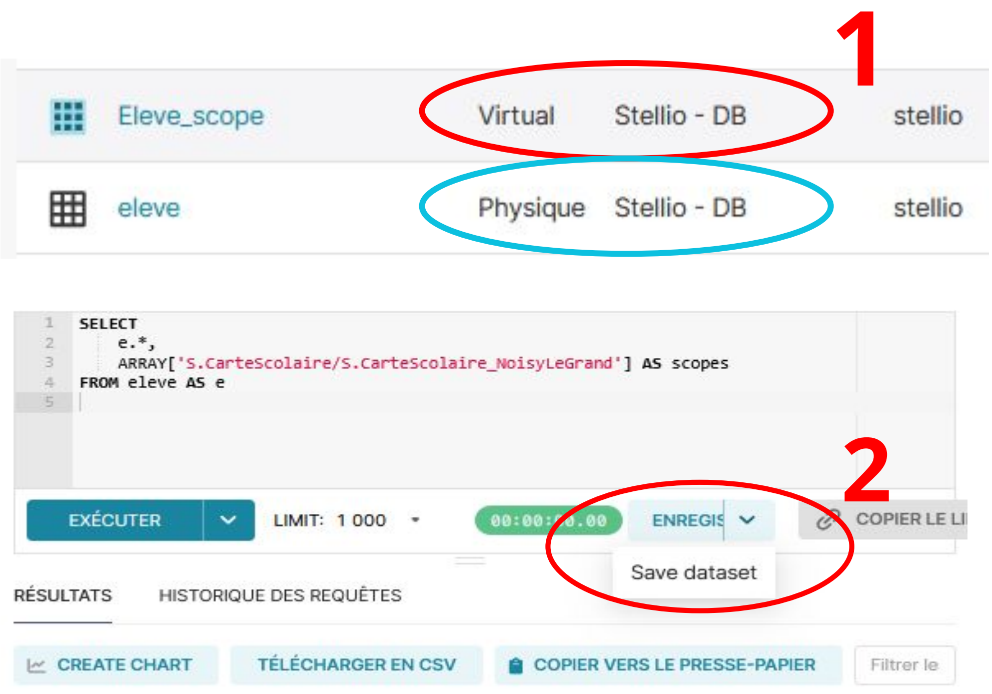
5. BI steps – create visuals

3. Prototyping

Energy Consumption in the City of Paris

1. Import data in SUPERSET
2. Create the virtual datasets
3. Start to create the dashboard
4. Understanding filters and connection

Once the **data is exported** to Superset, the next step is to **retrieve it and transform it into datasets** so that you can use it in your dashboards. By **following the procedure for creating a new dataset, you can create a physical dataset**, that is, directly linked to the data exported from Stellio via NiFi. Superset also offers the possibility to **create virtual datasets**, built from your own SQL queries in the SQL Lab.



4. BI step – create visuals

These virtual datasets allow for further analysis by performing joins between multiple physical datasets, adding new columns or calculated fields, or even creating custom measures. This flexibility allows for a fine adaptation of the data to the specific needs of the visualizations and indicators of your use case.

LDT4SSC

5. BI steps – create visuals

Energy Consumption in the City of Paris

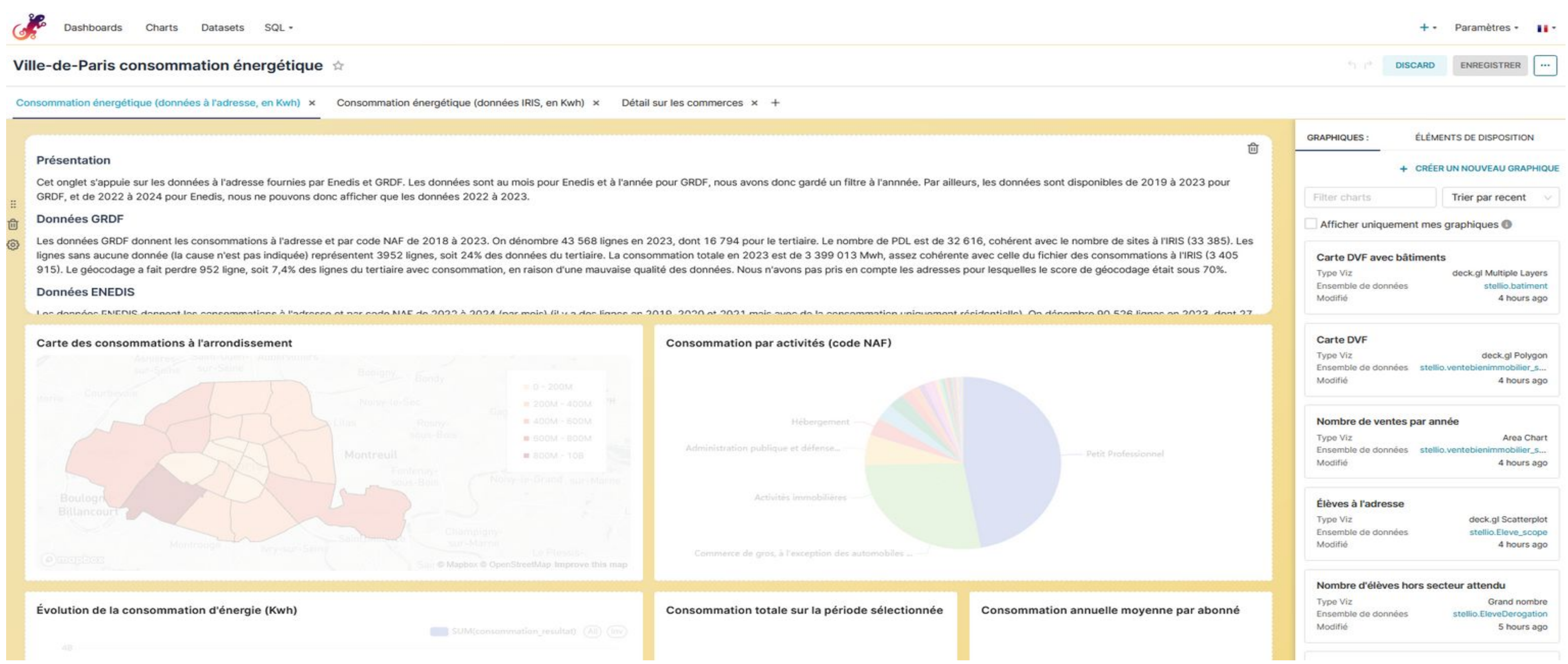
3. Prototyping

1. Import data in SUPERSET
2. Create the virtual datasets
3. Start to create the dashboard
4. Understanding filters and connection

Creating a dashboard in Superset is intuitive and based on the **same principles as most Business Intelligence tools**.

1) You need to **create charts** (graphs or visualizations) from the datasets you have prepared previously.
2) Once your charts are configured, you can **add them to the dashboard using a drag-and-drop system**, positioning them freely according to your desired layout.

Superset also allows you to **add interactive filters and link multiple charts together**, so that selections made on one graph are reflected on the others.



4. BI step – create visuals

The tool offers great customization freedom—choice of visualizations, filters, interactions, layout, etc.— but its functional richness goes beyond the scope of this guide. To delve deeper, it is advisable to consult the numerous tutorials available online as well as the official Superset documentation, which is very comprehensive and regularly updated.

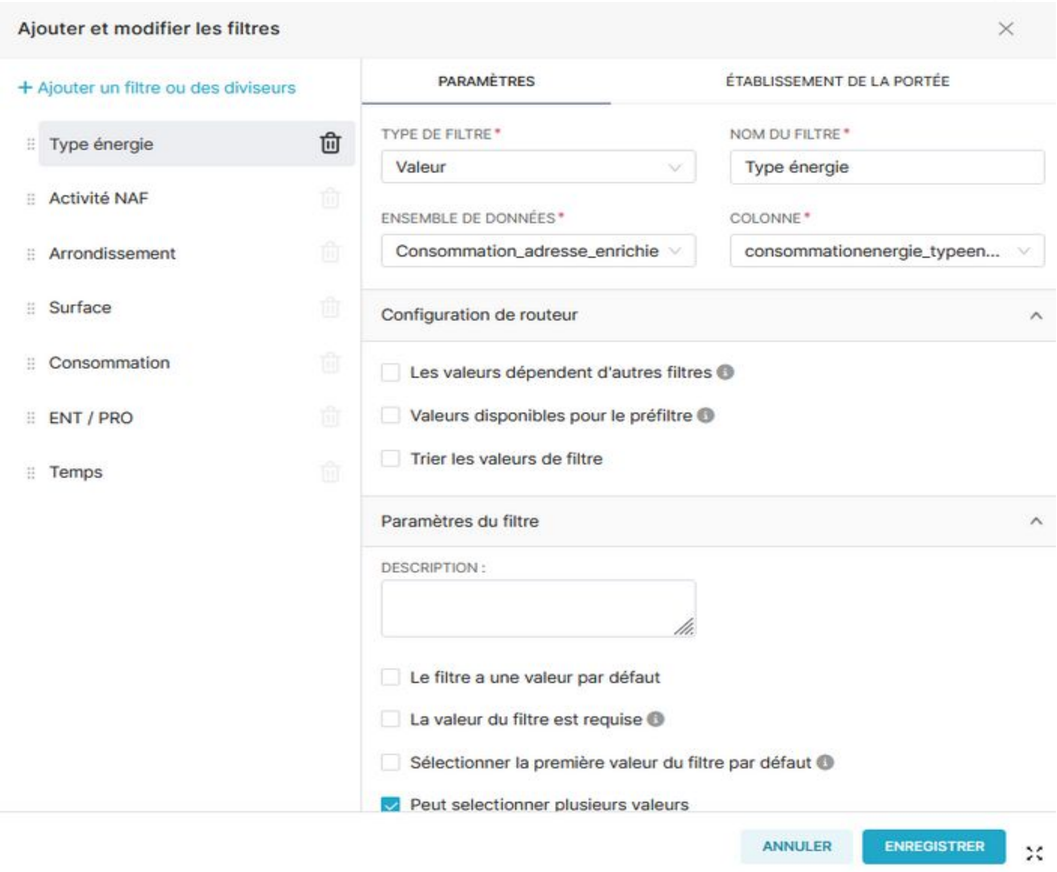
LDT4SSC

3. Prototyping

5. BI steps – create visuals

Energy Consumption in the City of Paris

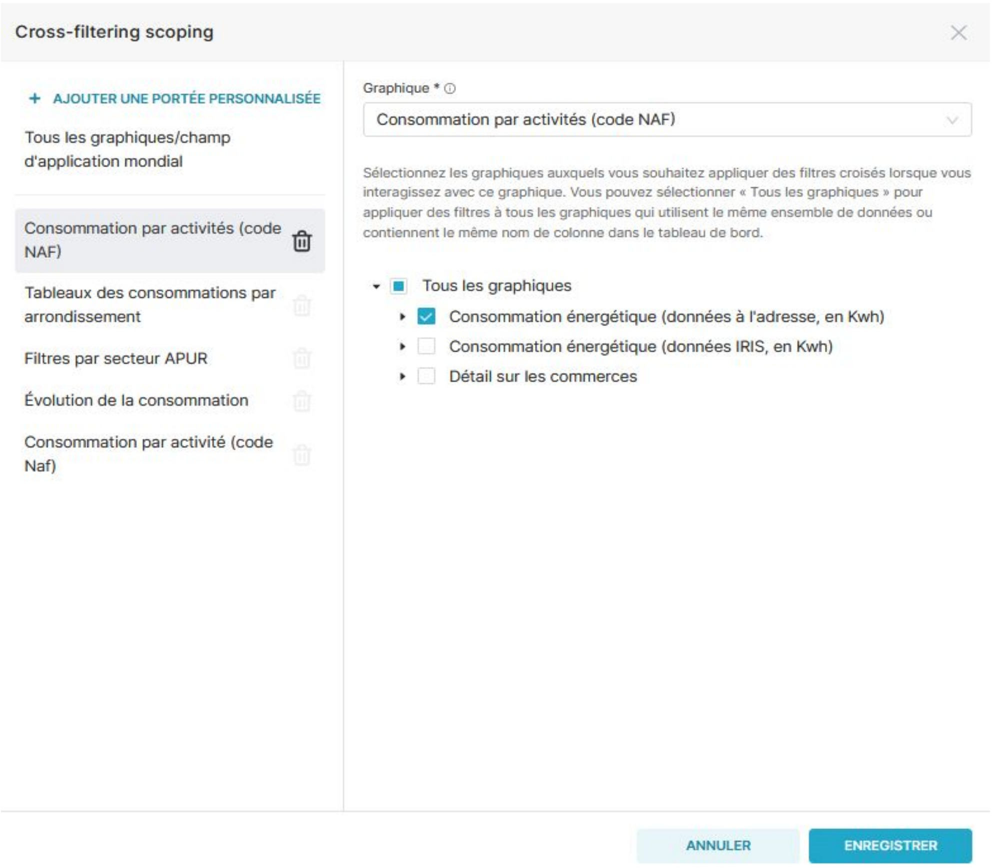
1. Import data in SUPERSET
2. Create the virtual datasets
3. Start to create the dashboard
4. Understanding filters and connection



Classic filters in Superset are easy to set up and can be configured directly from the **"+ Add / Modify Filters" interface**. They allow you to **restrict the data displayed in your charts** based on chosen criteria (for example, a period, a geographical area, or a category). These filters apply globally to the dashboard or the selected charts.

Cross-filtering, on the other hand, offers a **more dynamic interaction**: when a user clicks on an element of a chart (for example, a bar or a segment), it **automatically updates the other linked charts to display only the corresponding data**. This principle makes navigation smoother and more intuitive, allowing for interactive data exploration without having to manually manipulate multiple filters.

4. BI step – create visuals



Local Digital Twins for Smart Sustainable Communities

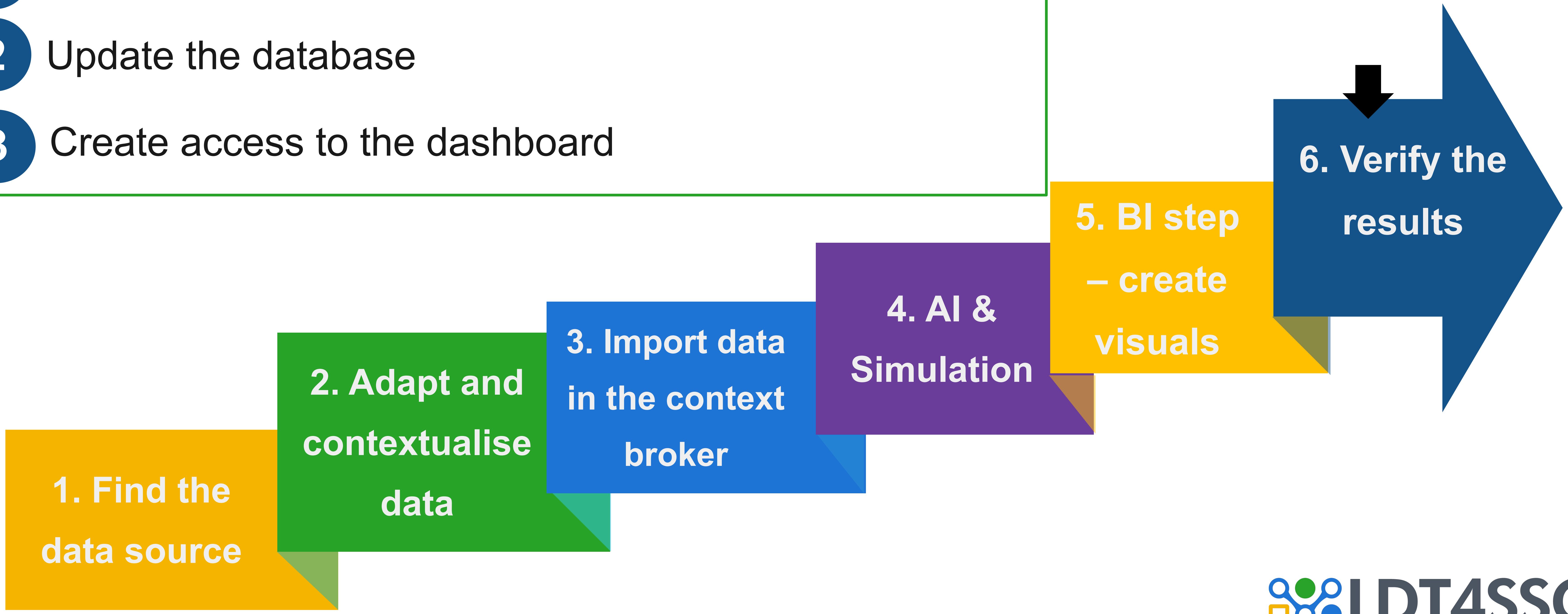
6. Verify results

Funded by
the European Union

6. Verify the results

Energy Consumption in the City of Paris

1. Check the consistency of the results and the source
2. Update the database
3. Create access to the dashboard



LDT4SSC

6. Verify the results

Energy Consumption in the City of Paris

1. Check the consistency of the results and the source
2. Update the database
3. Create access to the dashboard

This step involves **verifying the consistency between the source data and the data displayed in the final rendering**. The objective is to **ensure that no unjustified loss or alteration has occurred throughout the transformation and integration process**. If the data matches perfectly, it confirms the good quality of the flow and processing. On the other hand, **if discrepancies are observed** —for example, a different number of records— it is important to **be able to justify these differences**. This can come from geocoding that excluded poorly formatted addresses, the voluntary deletion of fictitious data to ensure the reliability of the final dataset, or even filters applied during cleaning. This control phase ensures the traceability and transparency of the entire use case production process.

5. Verify the results

LDT4SSC

6. Verify the results

Energy Consumption in the City of Paris

3. Prototyping

1. Check the consistency of the results and the source
2. Update the database
3. Create access to the dashboard

5. Verify the results

Once all the work is completed, it is recommended to re-execute the various NiFi flows associated with the use case. This step **allows for the validation of the proper integration of all the modifications made**, whether it involves adding variables, adjusting transformations, or updating the context. The re-execution also **ensures that the entire process runs smoothly**, from data ingestion to their display in the final rendering. It constitutes a **final check** for consistency and performance, essential before considering the use case as finalized and ready to be presented or put into production.

LDT4SSC

6. Verify the results

Energy Consumption in the City of Paris

3. Prototyping

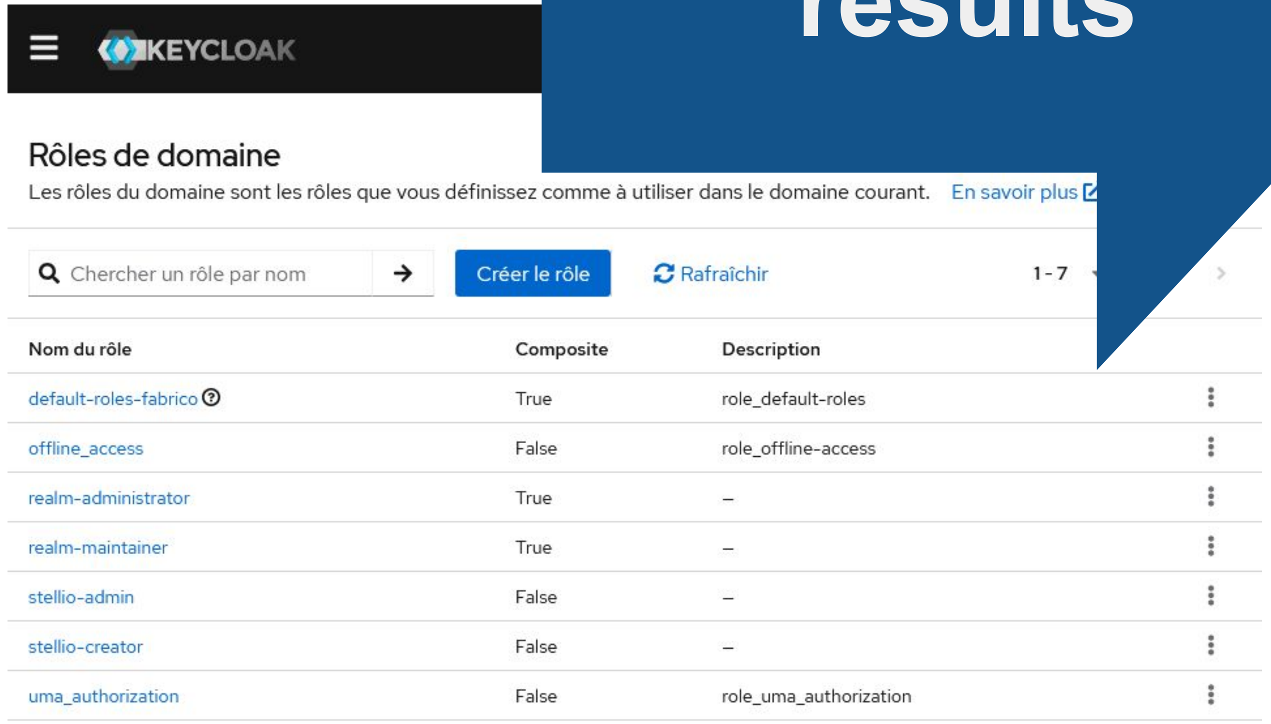
1. Check the consistency of the results and the source
2. Update the database
3. Create access to the dashboard

This clear structuring of roles ensures a controlled environment, where each actor has the rights appropriate to their responsibilities.

5. Verify the results

The final step in **creating a use case is to define the roles and access levels**. This phase is essential to ensure the **security and good governance of the system**. Role management is generally organized on **three levels**, taking into account the rights assigned across all system tools (NiFi, Stellio, Superset, Grafana, etc.).

1) The first level corresponds to **administrators**, who have all the **rights to read, write, and modify flows and data**;
2) The second level includes **contributors**, who are **authorized to view and manipulate data or modify certain flows**, without complete access to the overall configuration;
3) Finally, the third level concerns **end-users**, who **only access visualizations and dashboards** without the ability to alter the processes.



Going Further (1/2)

The development of AI brings additional challenges

The methodology is consistent and robust for its stated objective: **connecting heterogeneous data sources to a context broker (Stellio/NGSI-LD), normalizing them, and generating BI visualizations.** It is a classic interoperability pipeline designed for human consumption of existing data.

In this example, the method allows for the visualization of consumption trends, but **there may be other uses for the data** beyond simple visualization:

- predicting peaks,
- detecting anomalies,
- automatically optimizing settings.

This would **require artificial intelligence**, which needs data to be structured differently. Doing this in the visualization layer would be very static with “hard-coded” rules, whereas AI enables dynamic analysis.

The step 4 between the import into the context broker and the BI phase is therefore necessary, involving a “feature engineering” module to transform this data into explanatory variables (the “features”) relevant to the problem at hand. For example, to account for the fact that a measurement on a Monday in January has a different context than one on a Sunday in August.

A scheduled task (“training pipeline”) with a feedback loop back to the data would run regularly to maintain the learning process.

Going Further (2/2)

The development of digital twins also brings additional challenges

In this example, the method allows you to visualize consumption trends, but there may be other uses for reuse beyond simple visualisation with a LDT or a hypervisor.

- **Physical simulation/modeling:** the context broker synchronizes the real-world state, but a LDT must be able to run simulation models (“what happens if I shut down this network?” or “what will consumption be at 6 PM if the temperature rises by 3°C?”). This requires a simulation engine decoupled from the broker.
- **Bidirectionality: the method is read-only:** data → broker → dashboard. A true LDT also allows for feedback to physical systems (instructions to a controller, alerts to an operator with recommended actions). The “Step 4” would be needed for control/actuation flows.
- **Multi-layer temporality:** a LDT integrates three time horizons: past (historical data for BI), present (real-time via the broker), future (prediction/simulation). The method covers only the first two, and even then, real-time is implied rather than explicitly designed.

Copyright Message

Funded by the European Union

Cerema
CLIMAT & TERRITOIRES DE DEMAIN